

# Bases de Datos

Clase 3: Diseño de Bases de Datos I

# Hasta ahora

Conocemos el modelo relacional, por lo que podemos comenzar a diseñar una base de datos. Pero, ¿sabemos si lo estamos haciendo bien?

**Un error en la modelación puede ser muy costoso!**

Por ejemplo: olvidar añadir una columna.

## Esquema original

```
Personas(id, RUN, nombre)
Clientes(id, RUN, nombre)

Copie * desde Personas
a Clientes
```

✓ **Funciona correctamente**

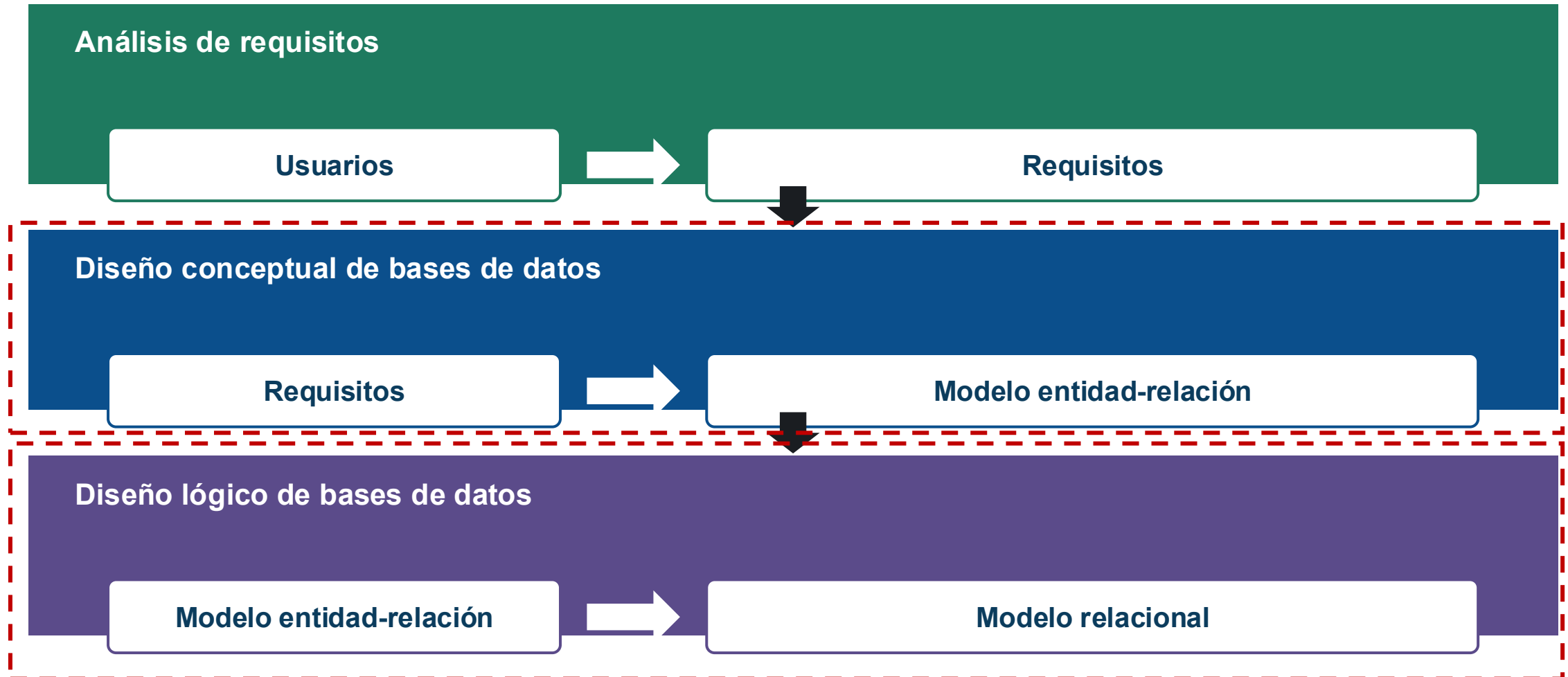
## Tras agregar una columna

```
Personas(id, RUT, nombre, edad)

Copie * desde Personas
a Clientes
```

✗ **Falla: Clientes no tiene columna "edad"**

# Diseño de base de datos



# Relational Database Management System (RDBMS)

Una base de datos relacional (RDB) es una forma de estructurar información en tablas, filas y columnas.

Una RDB tiene la capacidad de establecer vínculos entre información mediante la unión de tablas.

Un Sistema de administración de bases de datos relacionales (RDBMS) es un programa que se usa para crear, actualizar y administrar bases de datos relacionales.

# Construir una aplicación con un RDBMS

- En la práctica es imposible saber de antemano todos los requisitos que deberá cumplir un software.
- De la mano con eso, el esquema de la base de datos va cambiando a medida que progresa el desarrollo de un proyecto.
- Un esquema bien diseñado no solo permite consultar con facilidad y guardar los datos de forma óptima, sino que también permite modificarlo y aumentarlo con menos dolores de cabeza.

**Los errores en el diseño son muy costosos a la larga!**

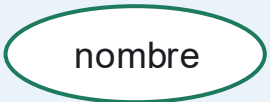
# Definiciones importantes

**Entidad:** Es un conjunto de objetos similares.



Producto

**Atributo:** Cualidades o propiedades de una entidad. Es lo que nos permite agrupar los objetos. Los atributos pertenecen a dominios, los cuales indican los posibles valores del atributo.



nombre

**Relación:** Es una asociación entre dos o más entidades.

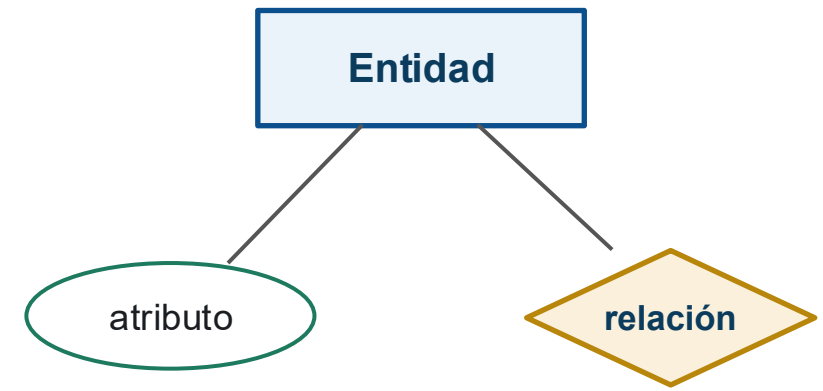


compra

# Diseño conceptual de la BD

## Ventajas de diseñar y diagramar la base de datos

- ✓ Definir entidades.
- ✓ Entender cómo se asocian esas entidades.
- ✓ Visualizar las restricciones del dominio.
- ✓ Lograr un diseño apropiado al problema.
- ✓ Mantener un esquema bien documentado.

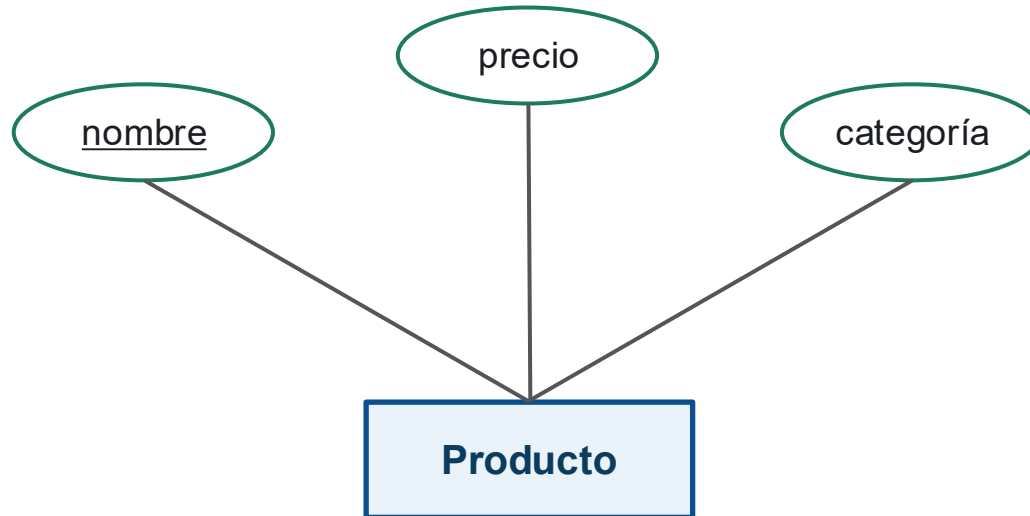


# Diagramas E/R



# Diagramas E/R

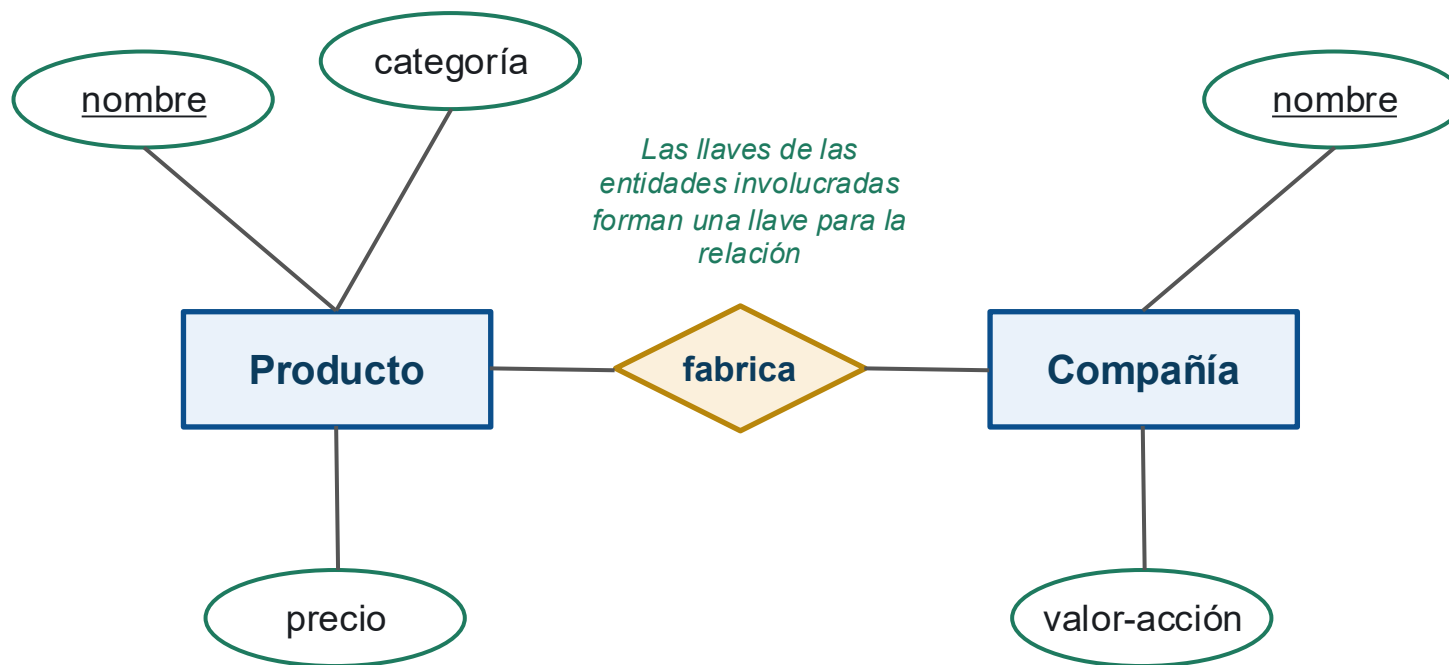
## Entidad con sus atributos

**Obligatorio:**

cada entidad debe tener un atributo llave.

**Llave:** un conjunto de atributos cuyos valores identifican de manera unívoca a cada entidad del conjunto. En el futuro denotaremos la llave subrayando el atributo.

# M. E/R → M. Relacional

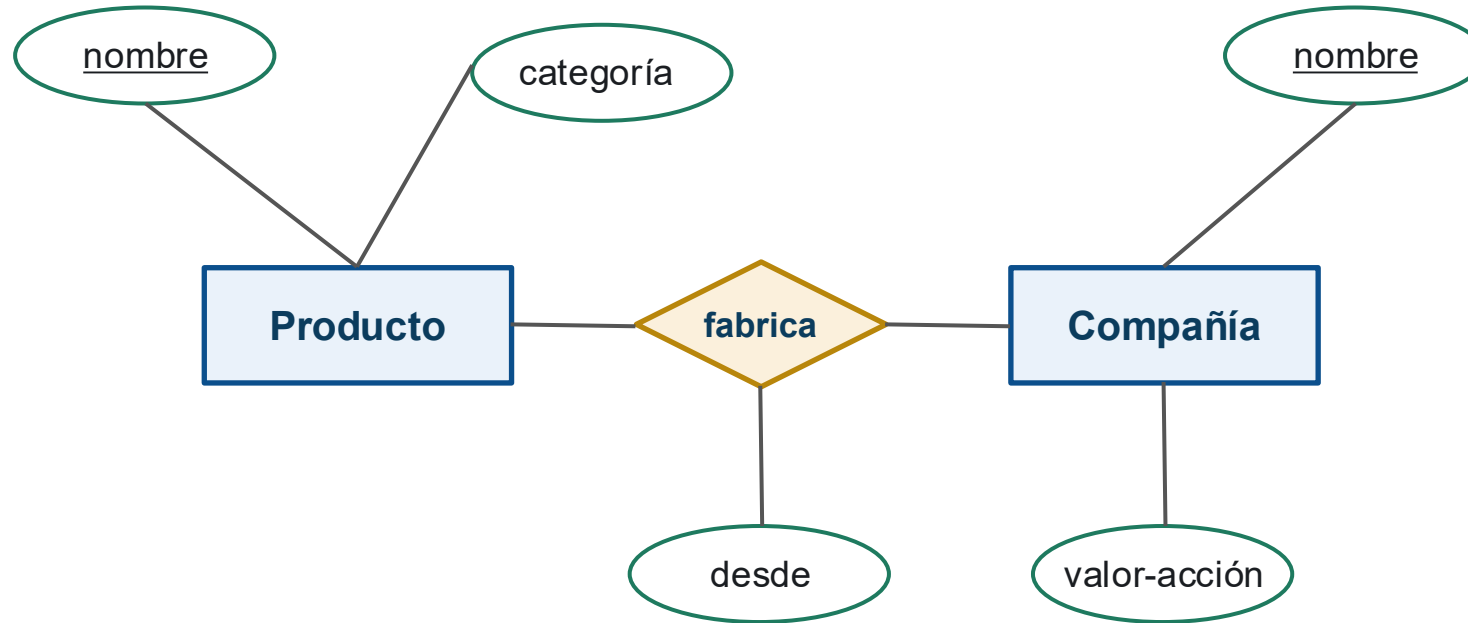


## Esquema

```
Producto(nombre: string, precio: int, categoría: string)
Compañía(nombre: string, valor-acción: int)
Fabrica(Producto_nombre: string, Compañía_nombre: string)
```

# Relaciones Binarias

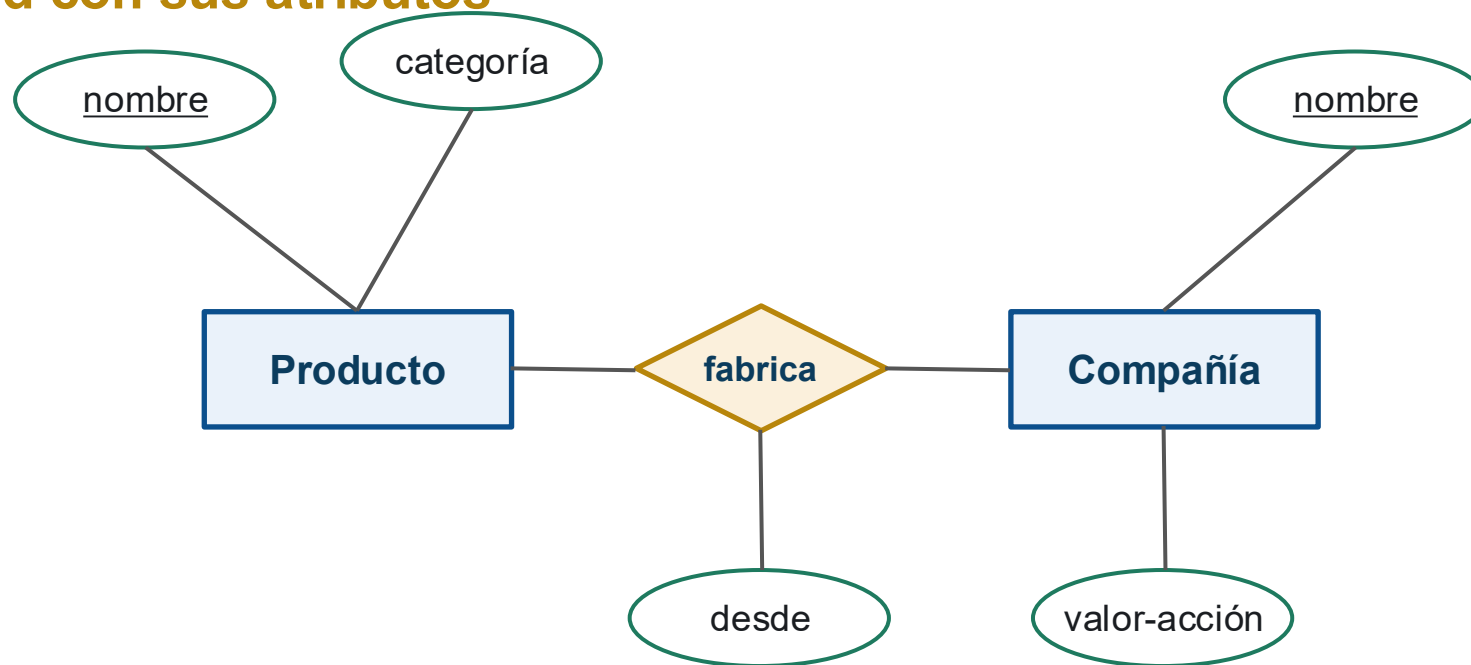
Las relaciones binarias son asociaciones entre **2 entidades**



Las relaciones también pueden tener atributos descriptivos que dan información de la relación.

# Relaciones Binarias

## Entidad con sus atributos



No puede ser parte de una llave!!

# Relaciones Binarias

## Multiplicidades

n a n



*Una compañía puede fabricar varios productos. Un producto puede ser fabricado por varias compañías.*

n a 0 o 1



*Un producto se fabrica a lo más por una compañía. Una compañía puede fabricar varios productos.*

0 o 1 a n



*Un producto puede ser fabricado por muchas compañías. Una compañía fabrica a lo más un producto*

0 o 1 a 0 o 1



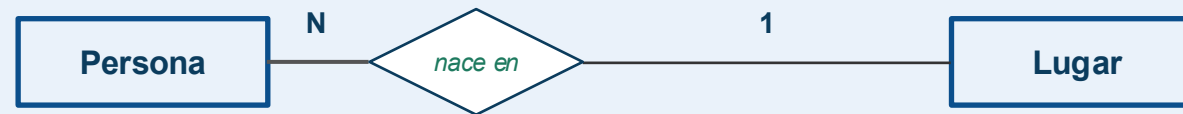
*Un producto se fabrica a lo más por una compañía. Una compañía fabrica a lo más un producto*

# Diferentes notaciones

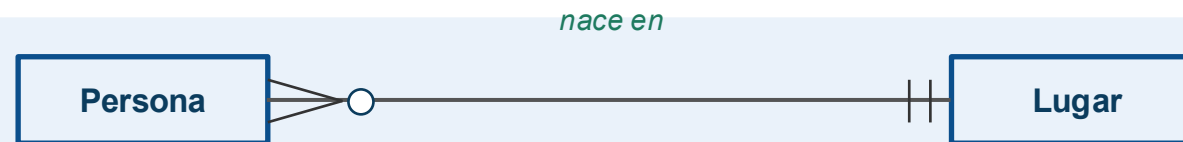
## Multiplicidades

Existen muchas convenciones distintas para representar la multiplicidad de una relación entre dos entidades.

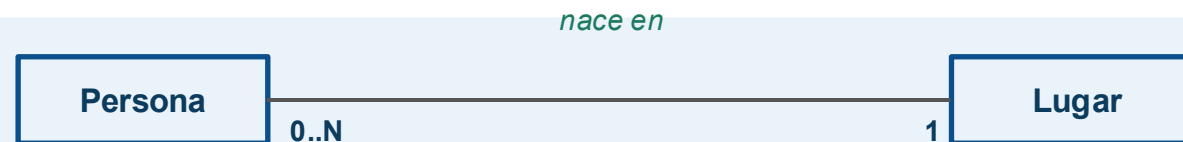
### Chen



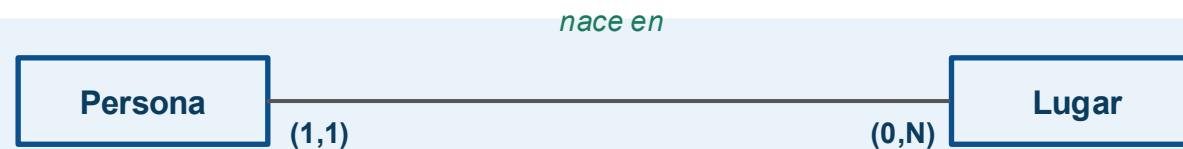
### Crow's Foot / Martin



### UML



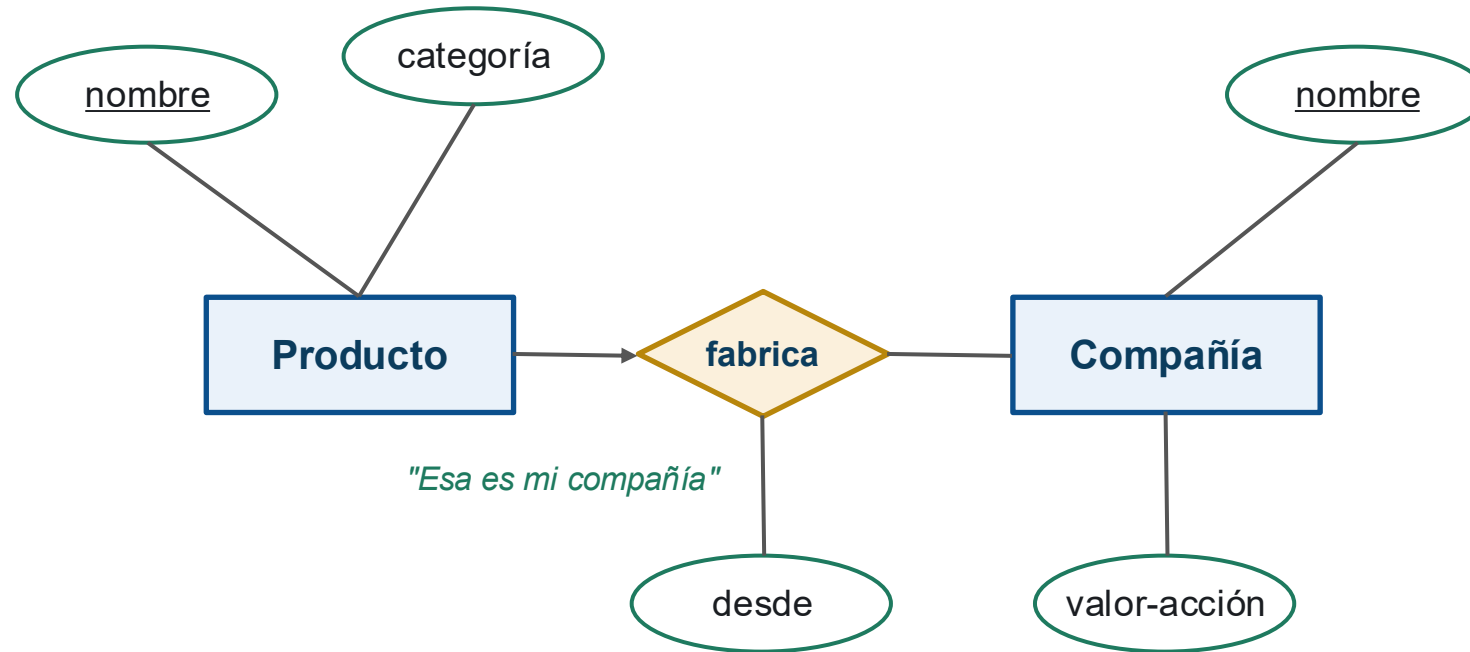
### Min-Max / ISO



*Nosotros usaremos siempre la convención de Ramakrishnan (flechas para indicar multiplicidad "a 1").*

# Relaciones Binarias

Sólo utilizaremos esta convención: Ramakrishnan



**Con la flecha:** la llave de fabrica es (llave de) Producto.

Multiplicidad de atributos es siempre "a 1".

# Diagramas E/R

## Relaciones Múltiples



# Relaciones Múltiples

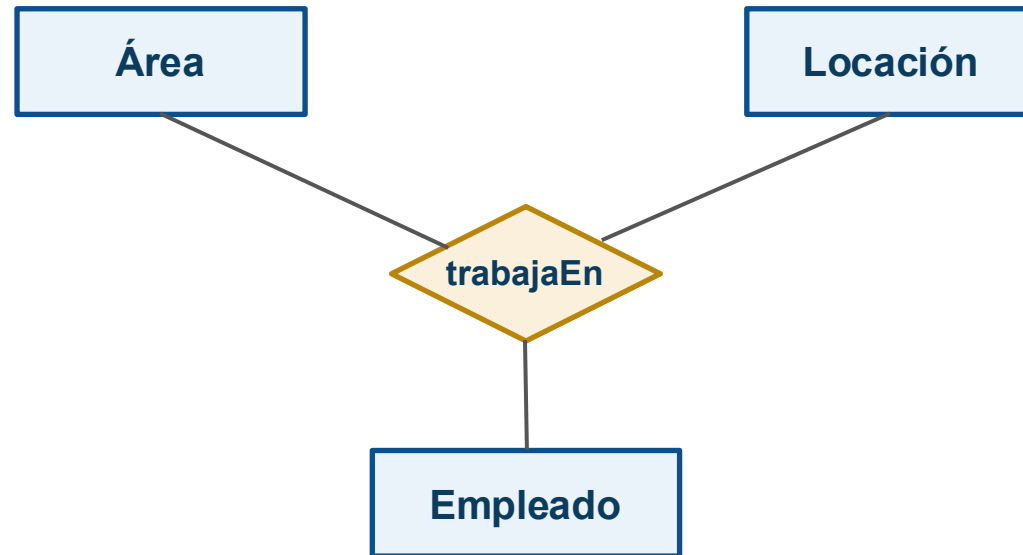
Una relación múltiple es una asociación entre **más de dos entidades**.

**PROBLEMA:** Una empresa tiene empleados y áreas en las que trabajan. Además, cada área tiene diferentes locaciones en las que se trabaja.

*Modelemos las locaciones y áreas en la que trabaja un empleado.*

# Relaciones Múltiples

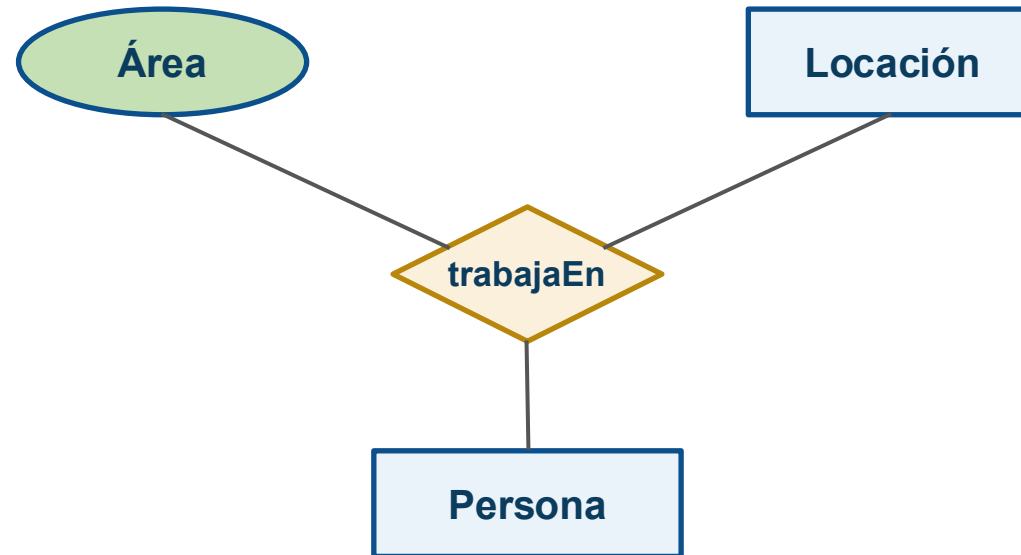
¿Cómo se puede modelar las locaciones y áreas en la que trabaja un empleado?



# Relaciones Múltiples

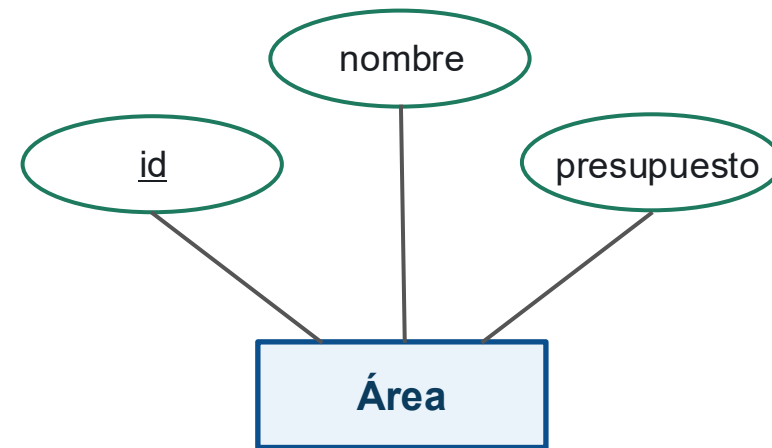
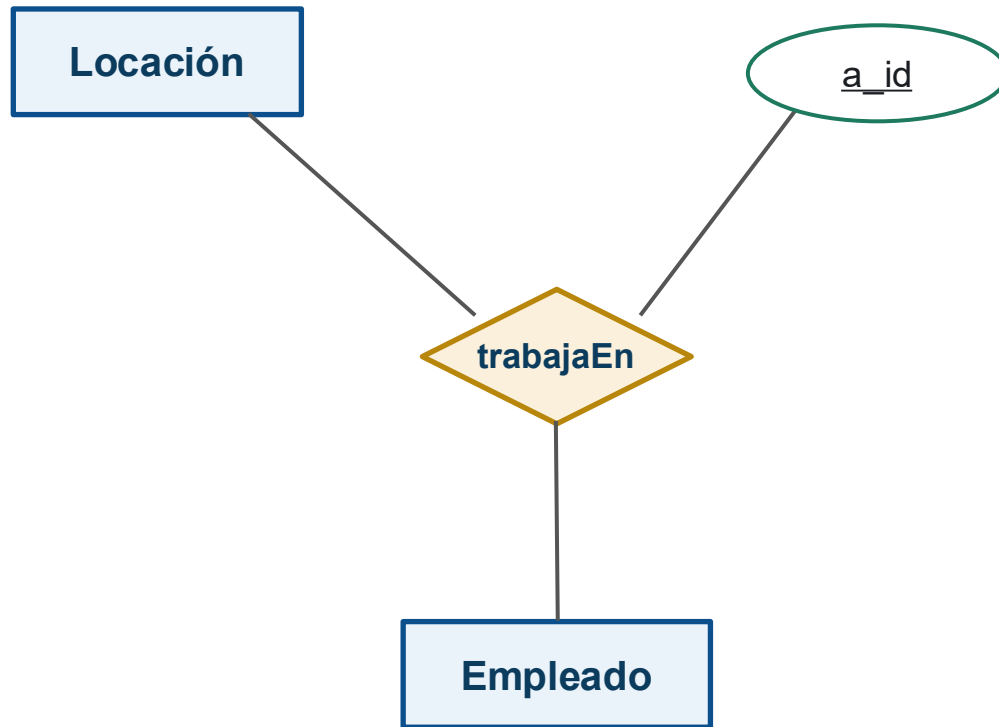
¿Por qué no un atributo?

Quizás no es un "valor simple": ¿qué pasa si queremos guardar año, presupuesto, etc?



# Relaciones Múltiples

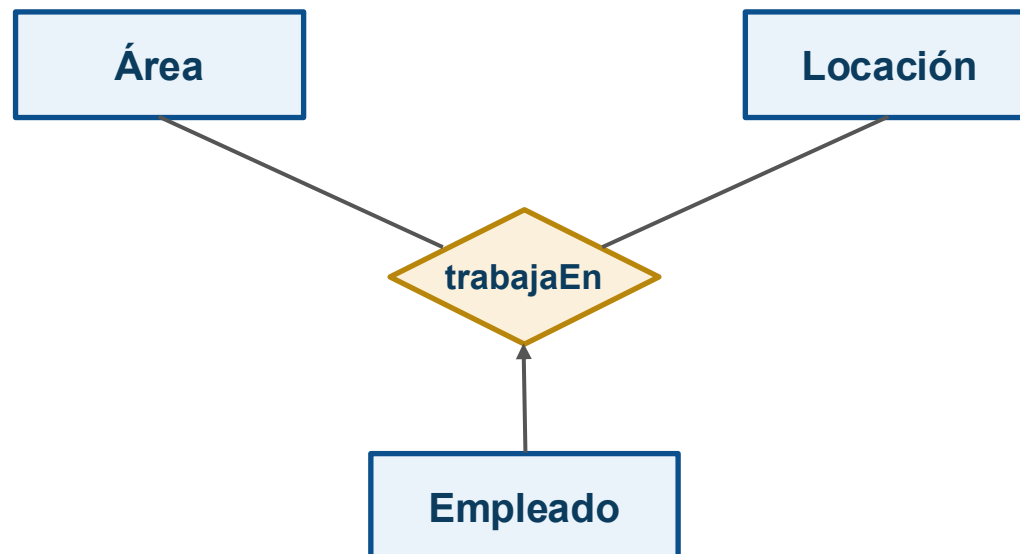
¿Y ahora?



No se pueden relacionar atributos de diferentes entidades.

# Relaciones Múltiples

¿Qué significa esto?

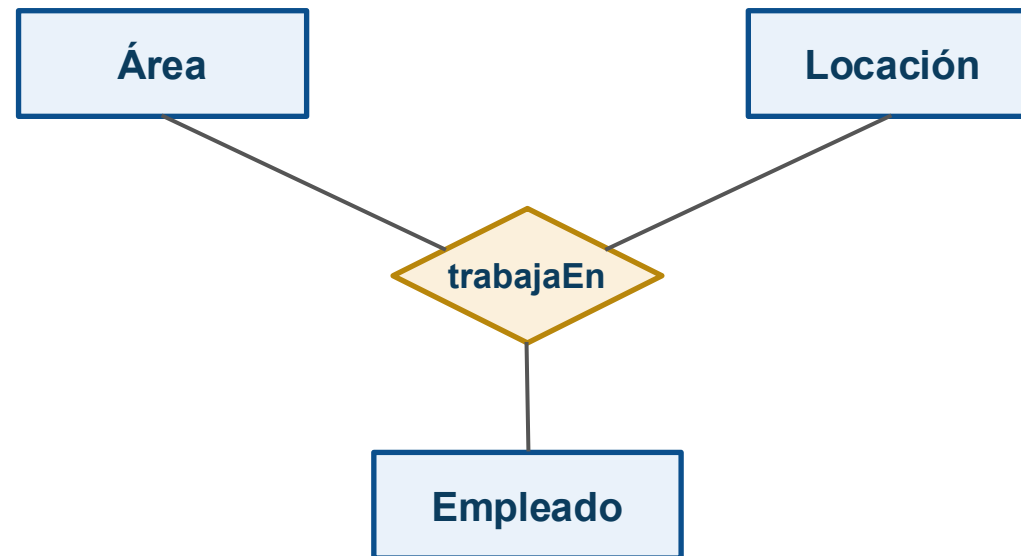


Un empleado puede trabajar a lo más 1 en una sola área y en una sola locación.

"Dime el empleado y te diré donde trabajas" (flecha = llave)

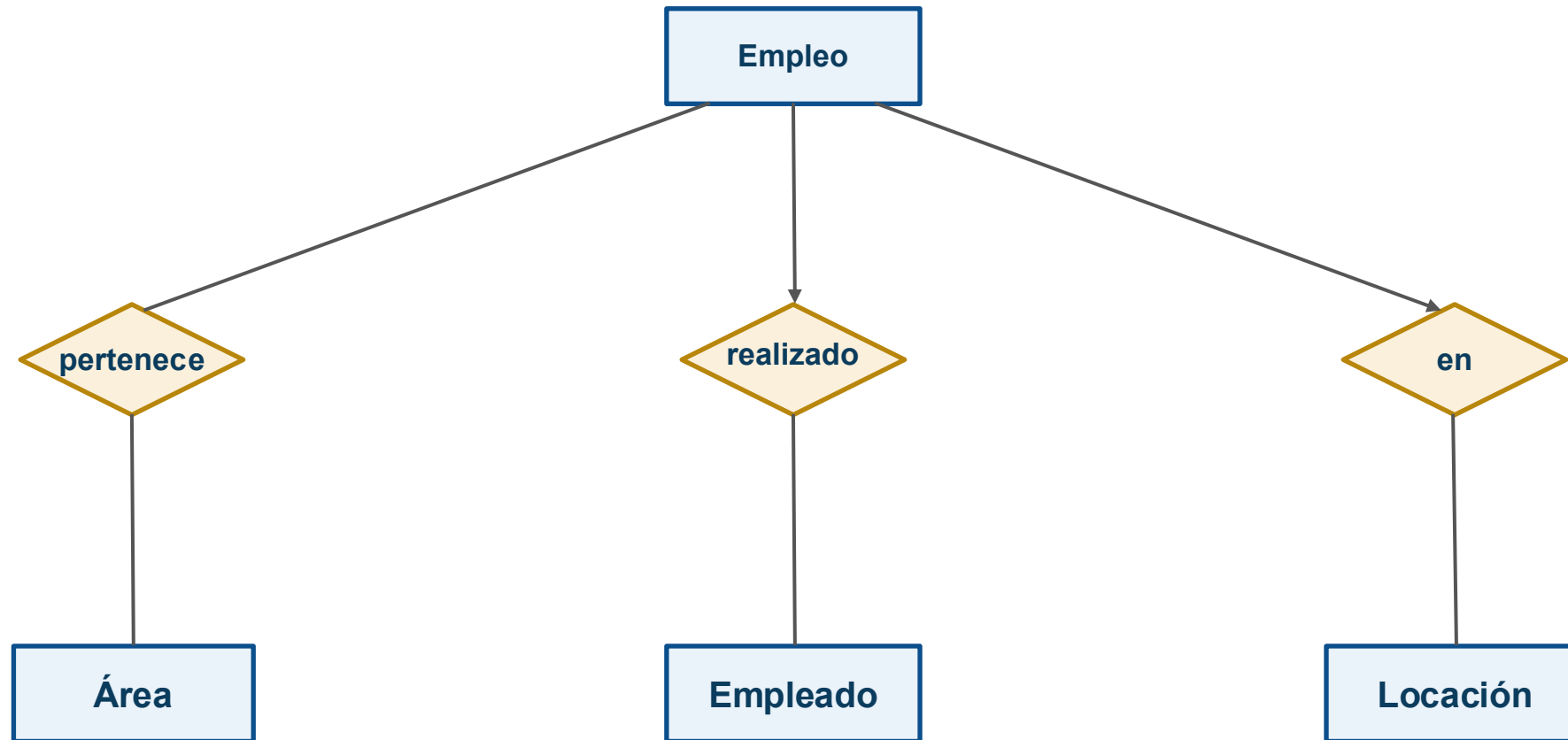
# Relaciones Múltiples

¿Y qué pasa si decimos que un empleado puede trabajar en varias áreas pero en una sola locación?



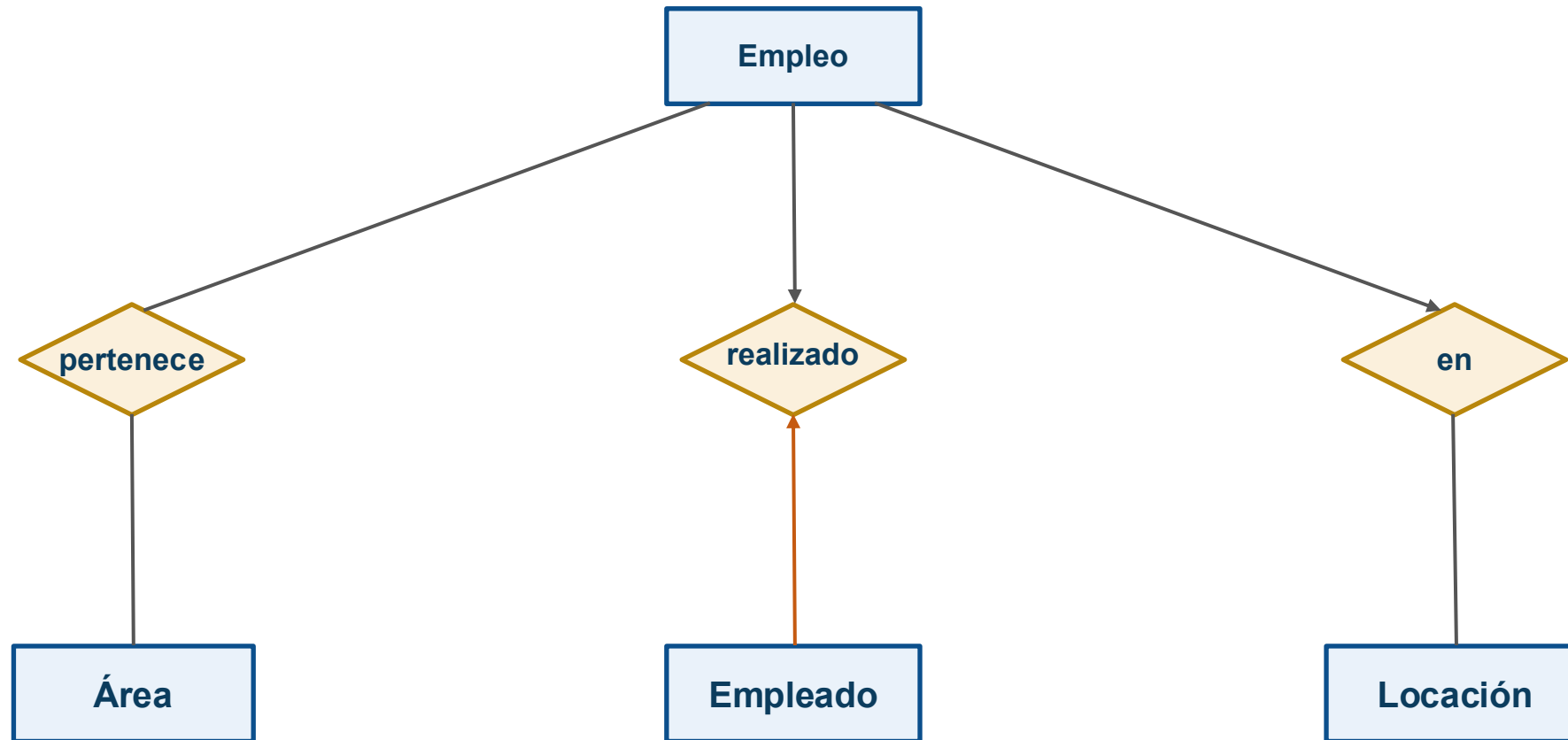
# Relaciones Múltiples

Si usamos solo relaciones binarias



# Relaciones Múltiples

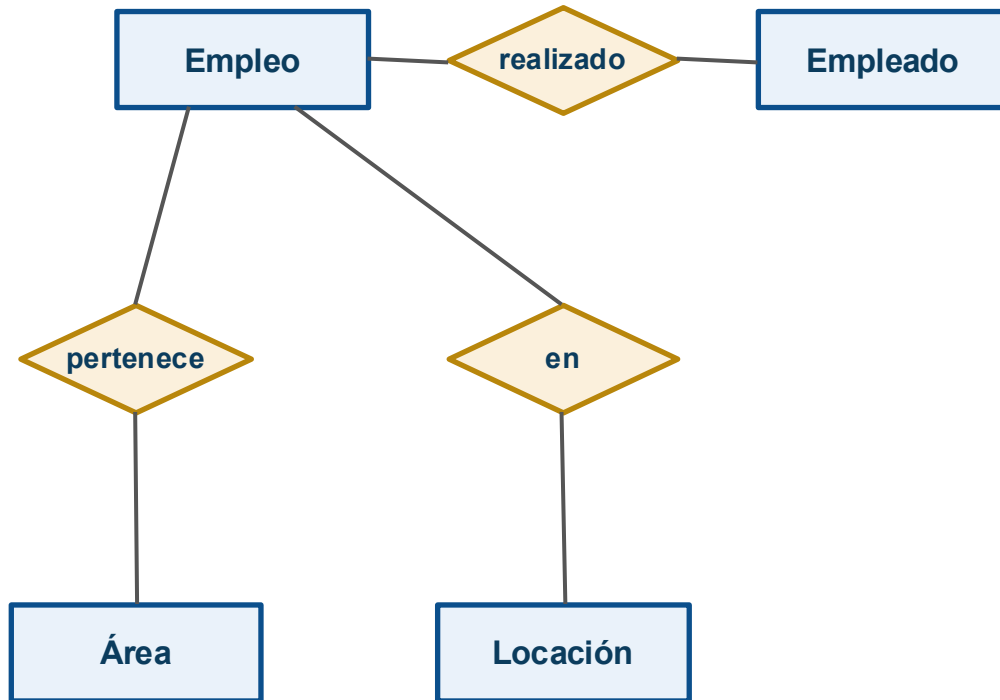
¿Y qué pasa si decimos que un empleado puede trabajar en varias áreas pero en una sola locación?



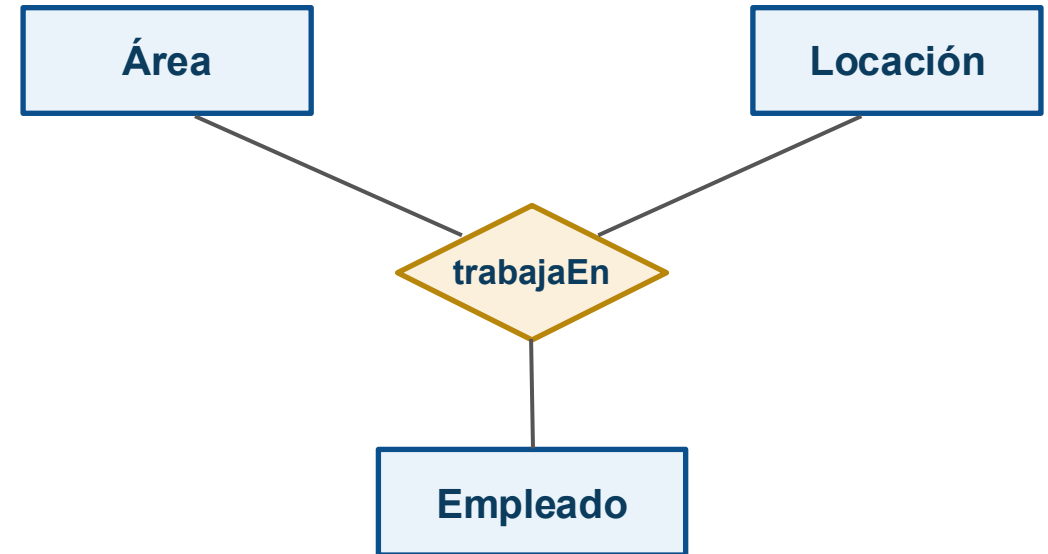


# Relaciones Múltiples

¿Cuál es mejor?



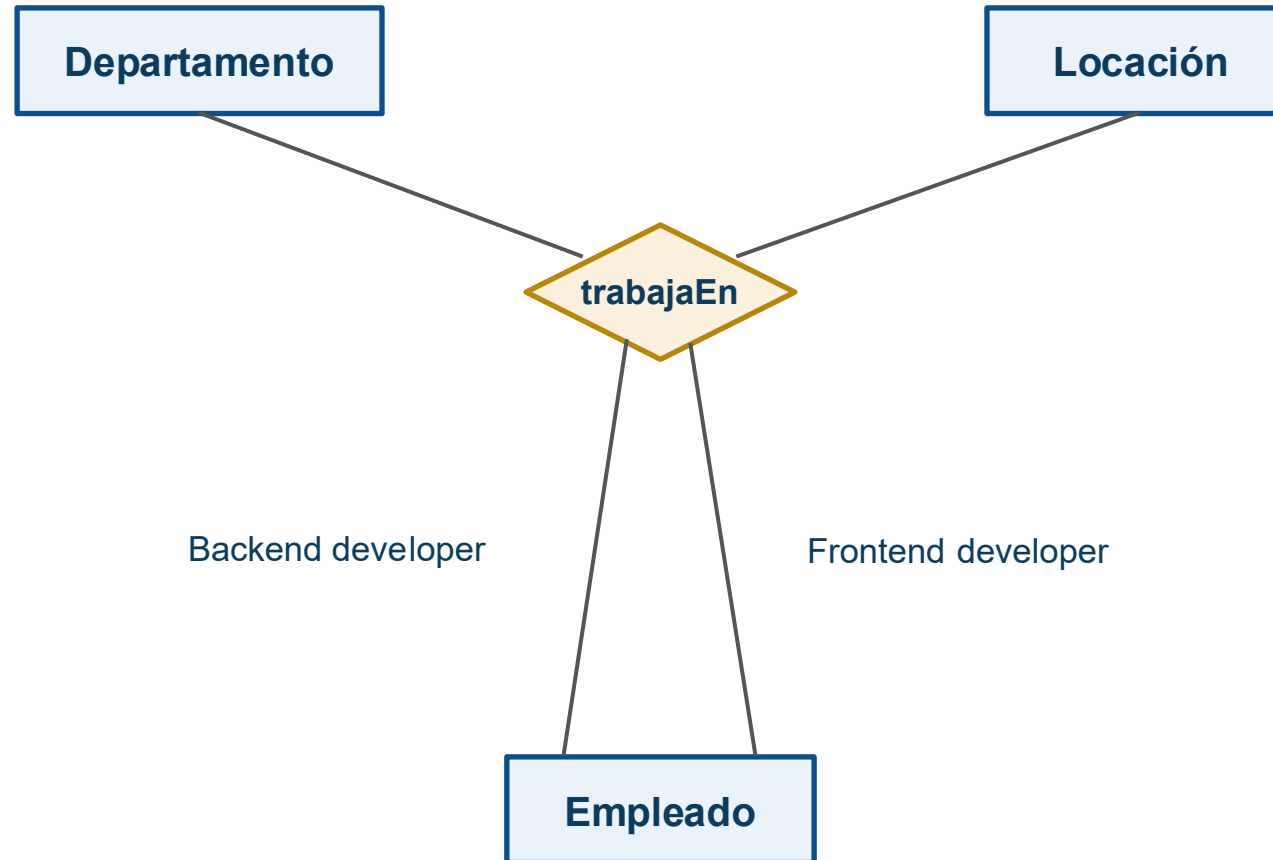
Más flexible



Más conciso

# Relaciones Múltiples

Una entidad puede participar más de una vez en una relación

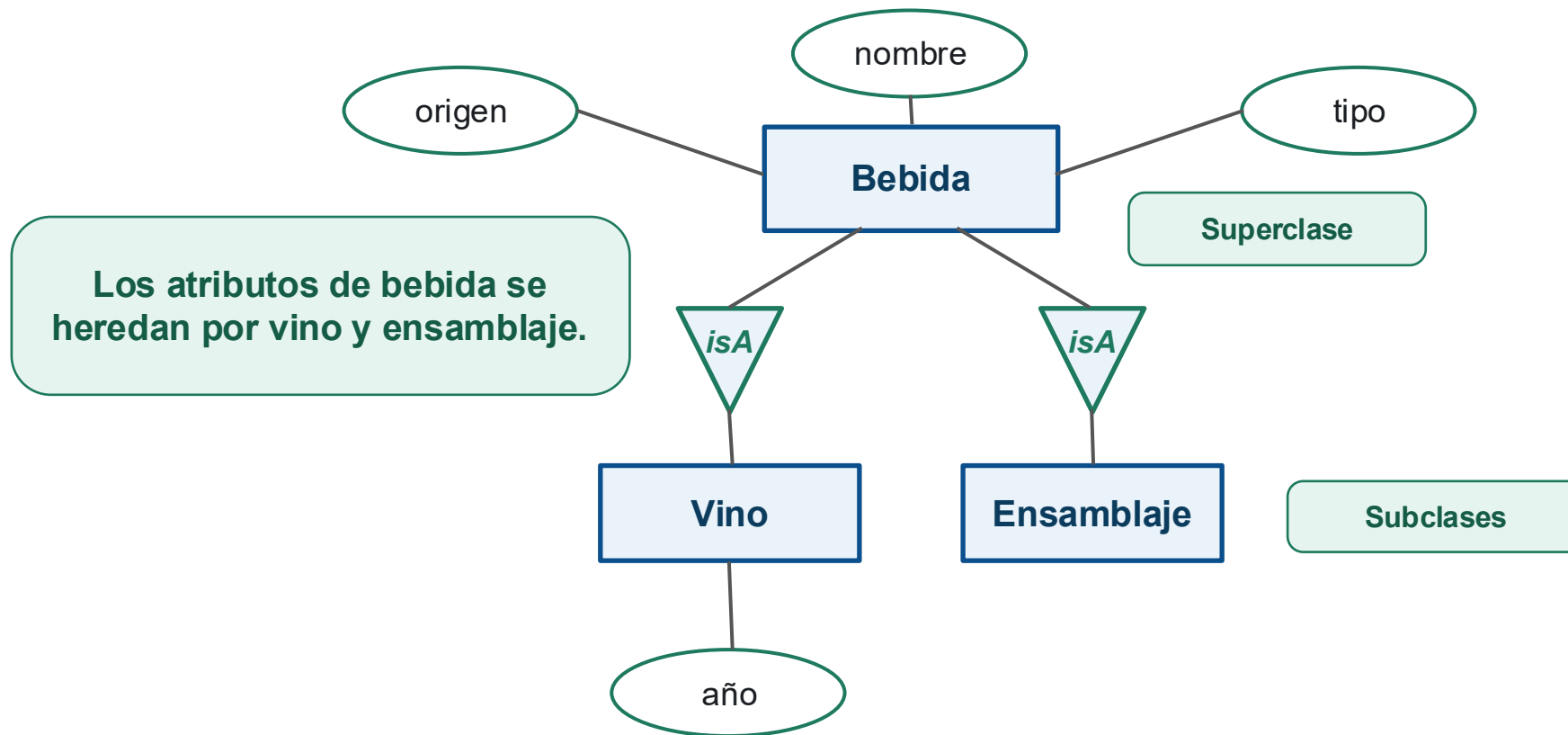


# Diagramas E/R

**Jerarquía de clases**

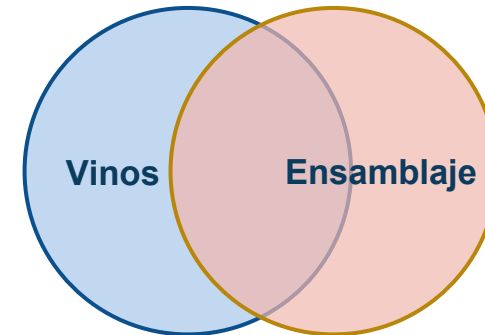
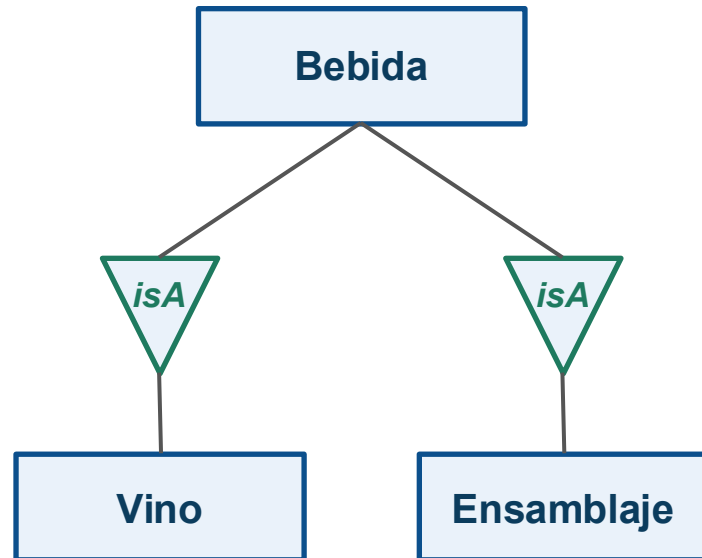
# Jerarquía de clases

**PROBLEMA:** Modelemos una distribuidora de licores o "Spirits".



# Jerarquía de clases

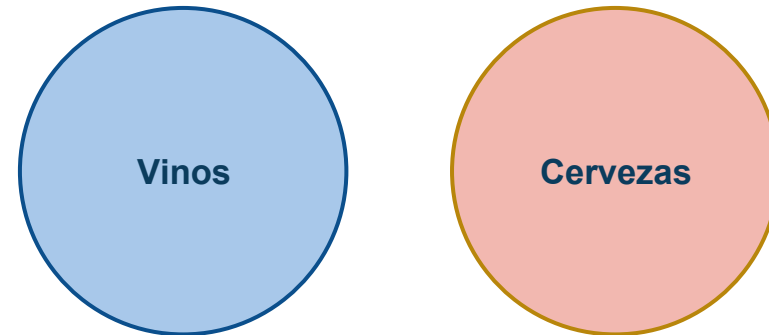
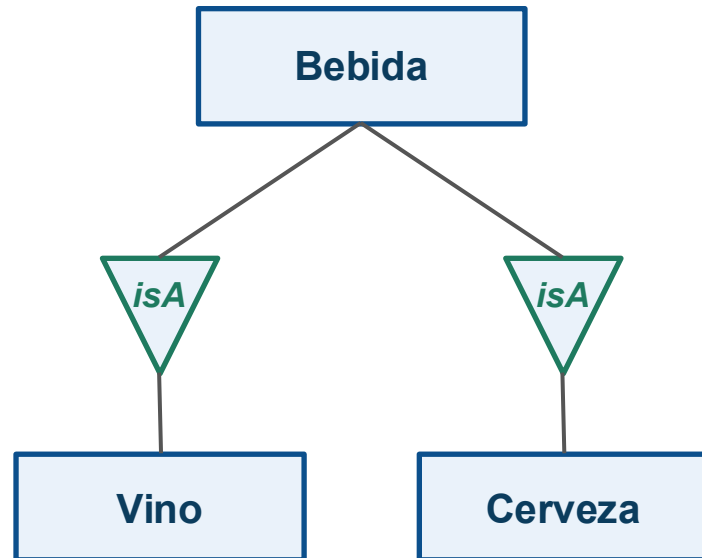
El solapamiento es una restricción que indica si dos subentidades pueden tener un mismo objeto



Los ensamblajes son un tipo de vino, por lo que acá sí hay solapamiento.

# Jerarquía de clases

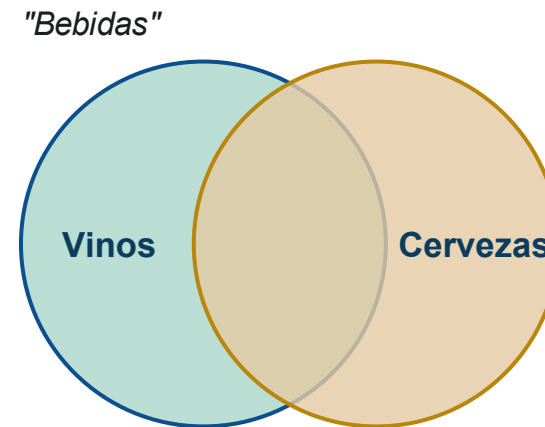
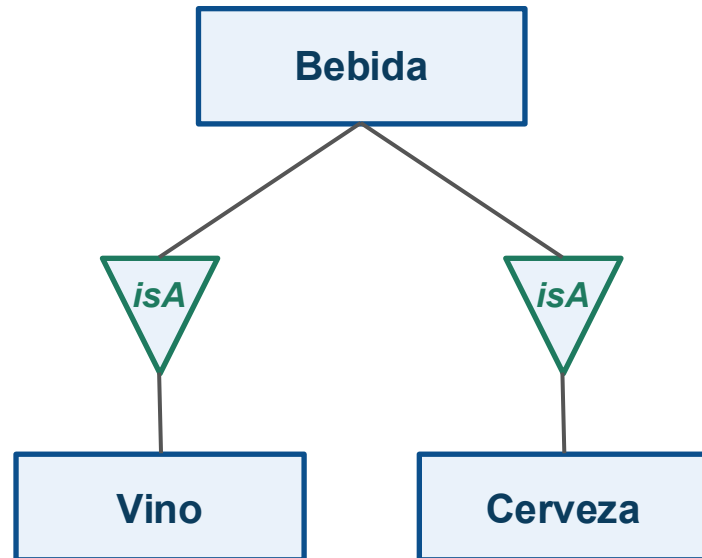
El solapamiento es una restricción que indica si dos subentidades pueden tener un mismo objeto



Acá no hay solapamiento. No hay "vinos acervezados".

# Jerarquía de clases

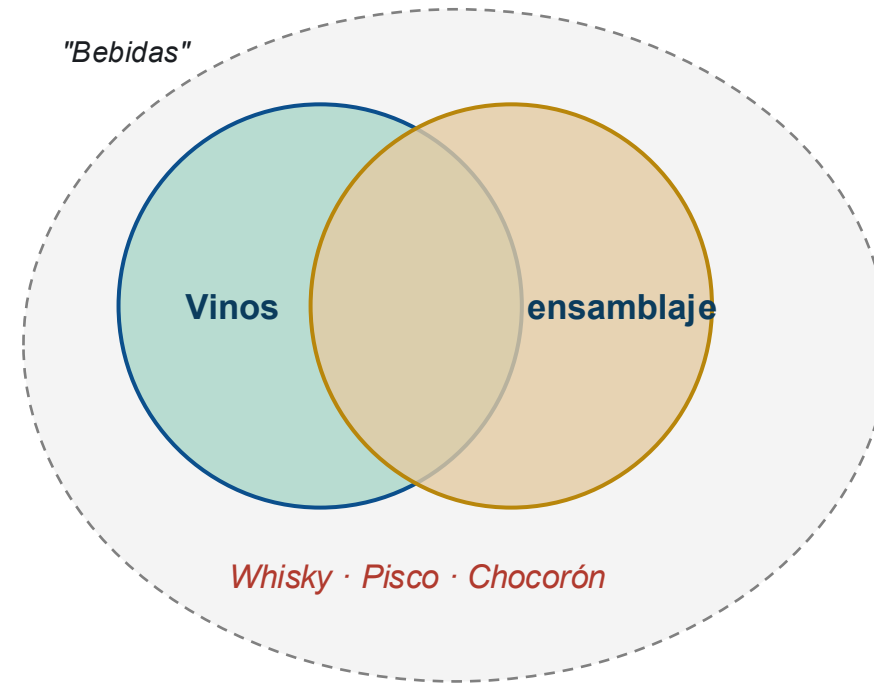
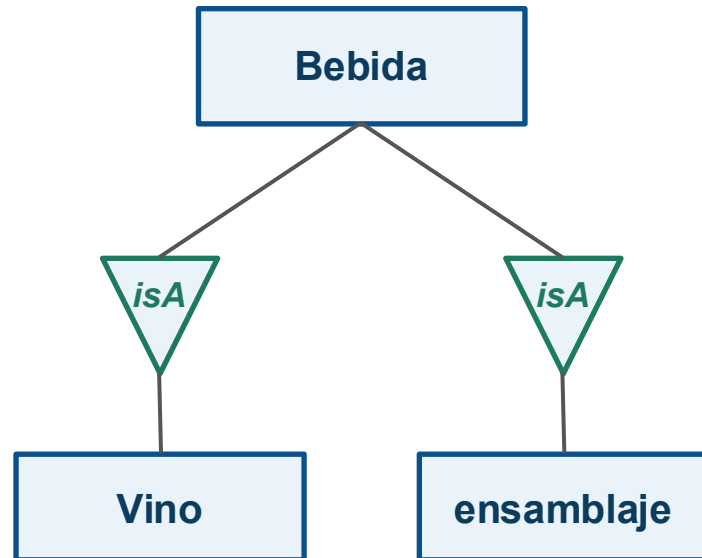
La cobertura es la restricción de que todas las sub-entidades estén en la super-entidad



Si tenemos un local que vende solo cerveza y vino, entonces se consideraría que está cubierto.

# Jerarquía de clases

La cobertura es la restricción de que todas las sub-entidades estén en la super-entidad



Si tenemos un local que además vende Pisco, Whisky y Chocorón, no se consideraría cubierto.



# Diagramas E/R

## Entidades Débiles

Una entidad débil es una entidad cuya llave depende de otra entidad.

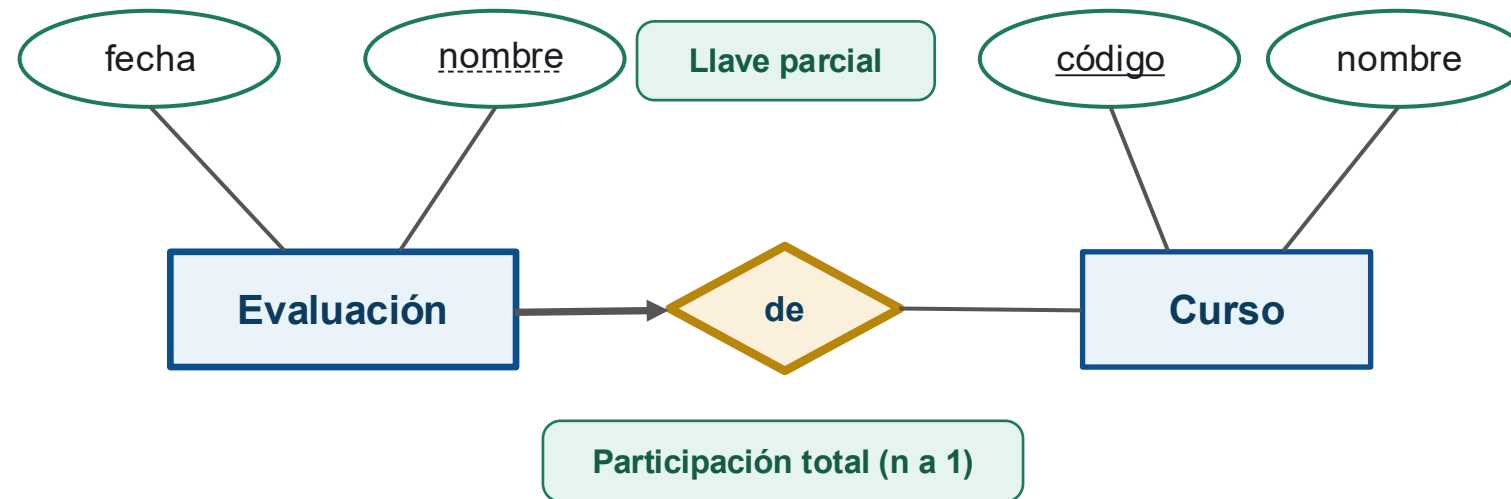
La entidad débil es identificada por sus atributos y por un atributo llave de otra entidad, la cual es llamada identifying owner.

La llave de la entidad débil es conocida como llave parcial.

# Entidades Débiles

## PROBLEMA: Modelemos un sistema de registro de notas

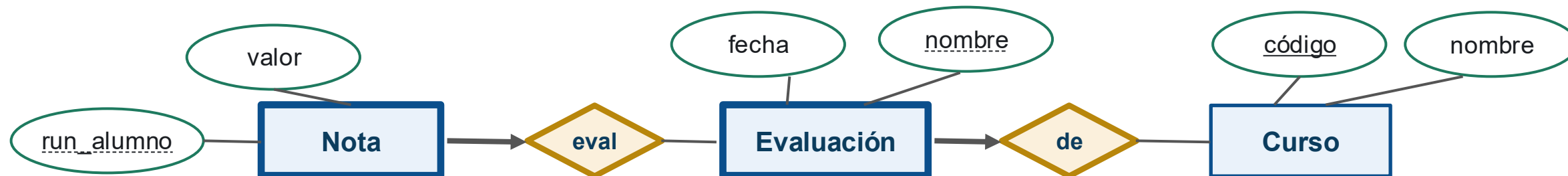
Como dijo Jack, "vamos por partes". Primero, veremos cuáles son las evaluaciones del curso.



La llave de Evaluación es el par (código, nombre).

# Entidades Débiles

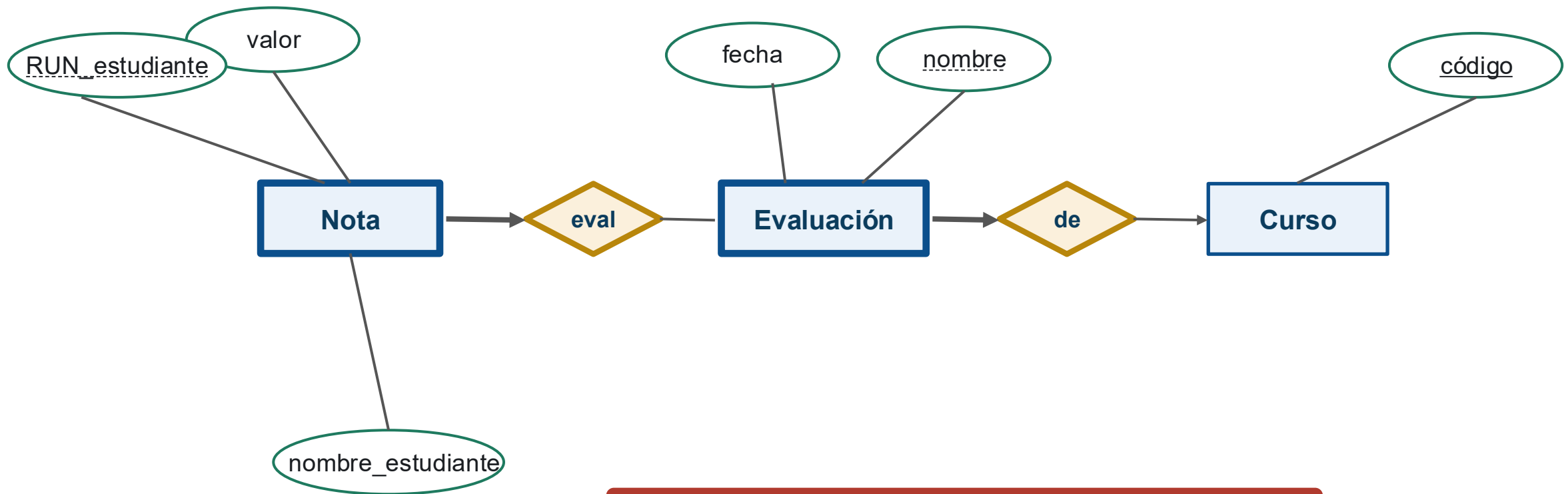
Ahora añadiremos las notas



La llave de Nota es la tupla (código, nombre, run\_alumno).

Falta el nombre del estudiante

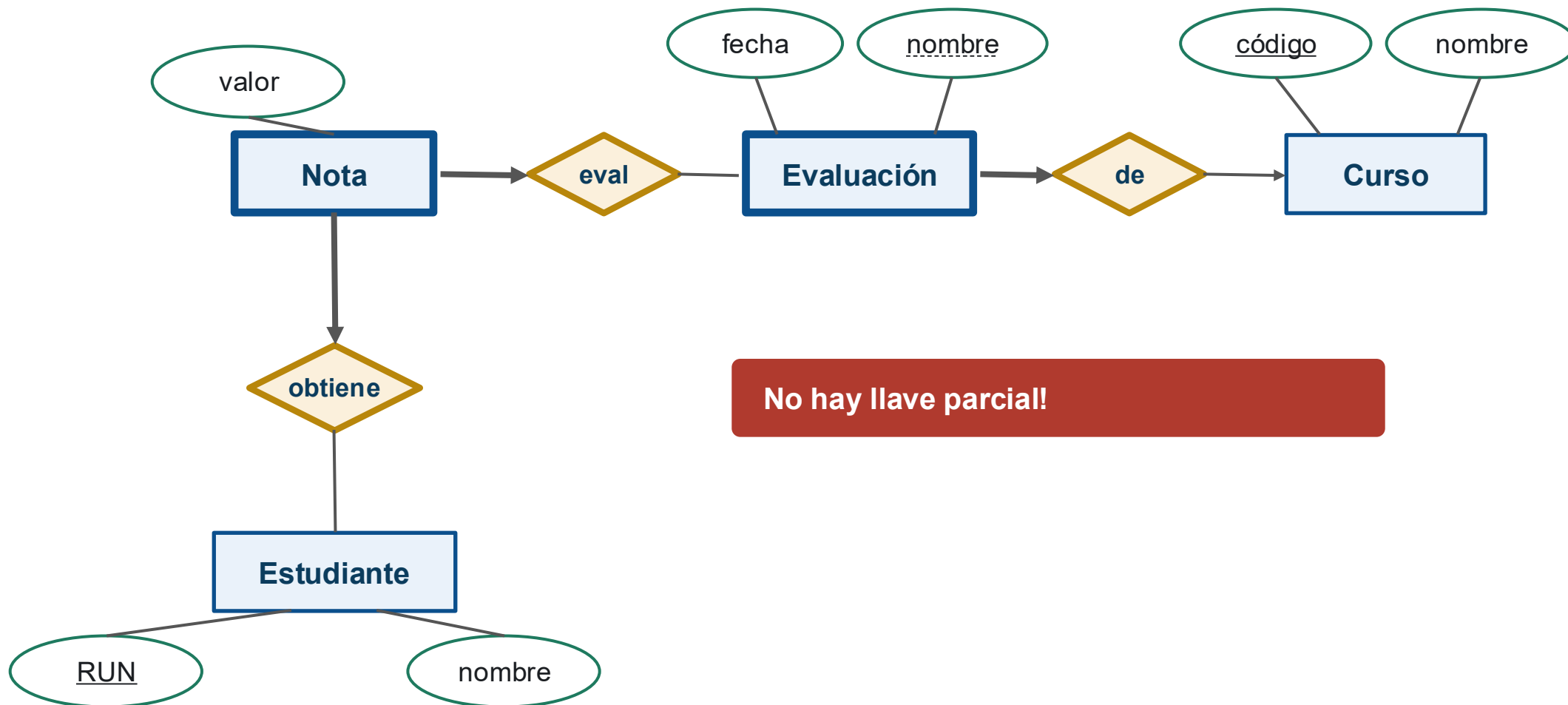
# Entidades Débiles



Problema: se repite el nombre del estudiante para cada nota (y es redundante con RUN).

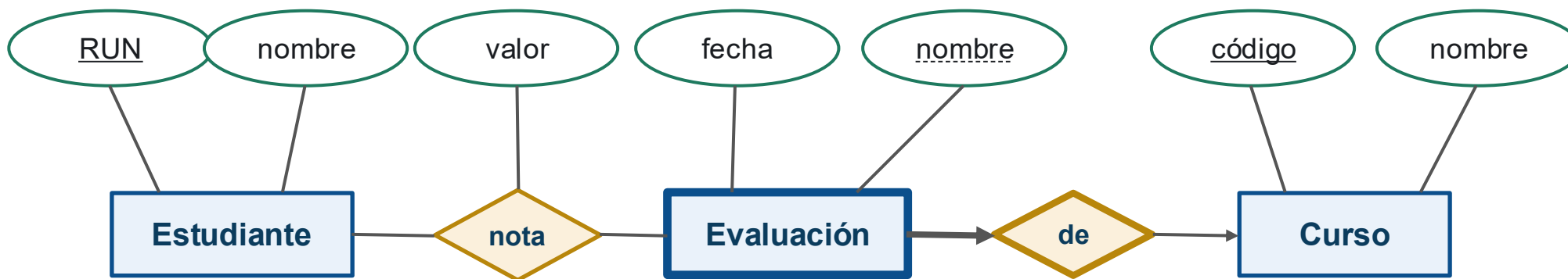
# Entidades Débiles

Se crea la entidad **Estudiante**



# Entidades Débiles

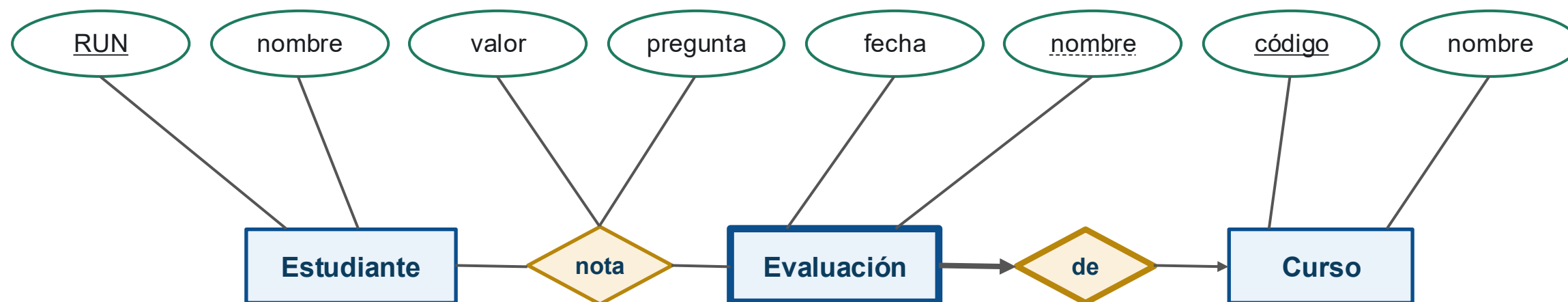
Nota pasa a ser relación



Incorporemos la nota por pregunta

# Entidades Débiles

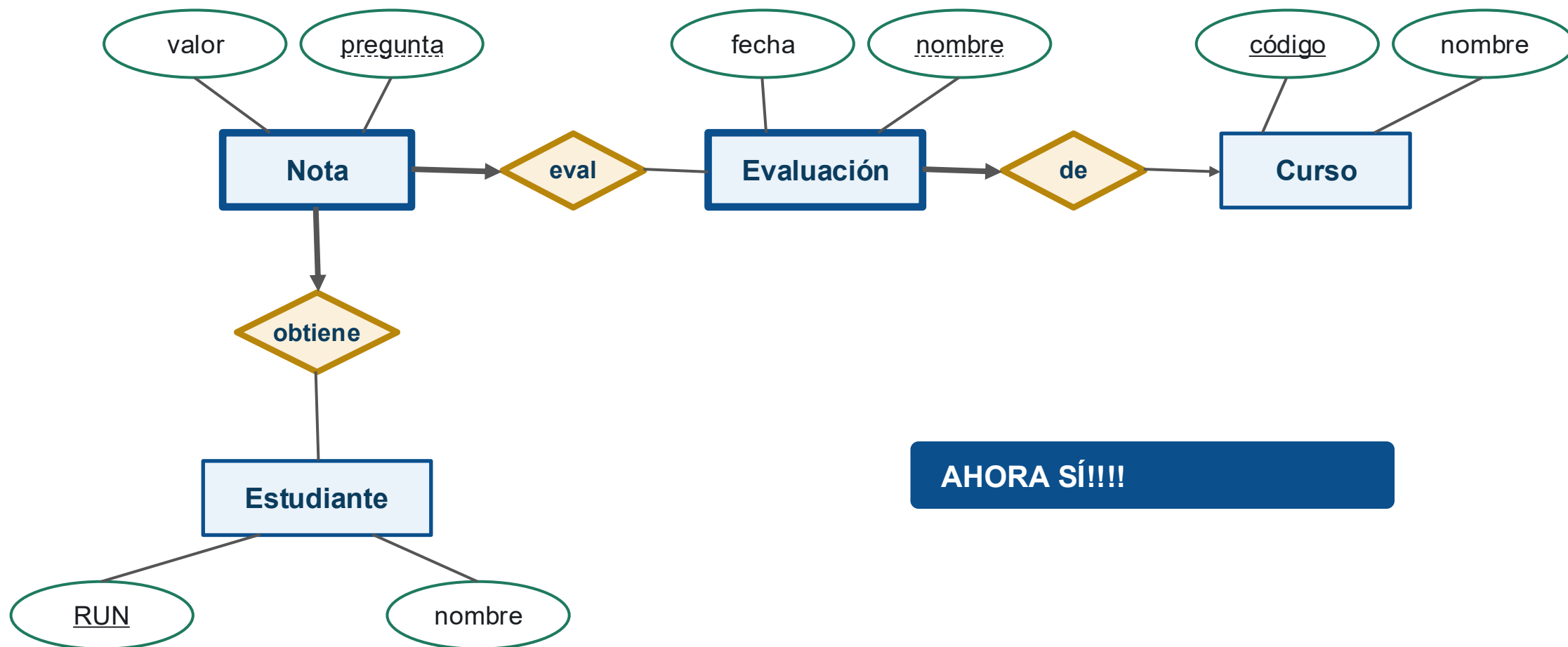
Se incorporan las notas por pregunta



¡Un estudiante puede tener varias notas en la misma evaluación!

# Entidades Débiles

## Nota por pregunta



AHORA SÍ!!!!

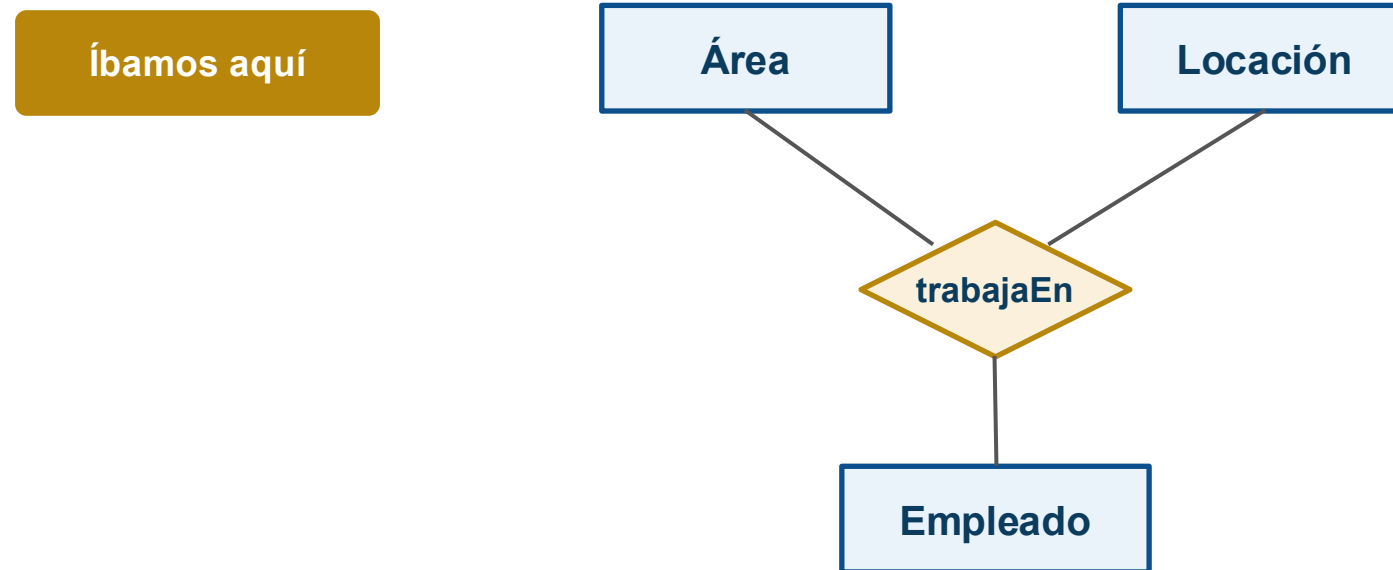


# Diagramas E/R

## Agregación

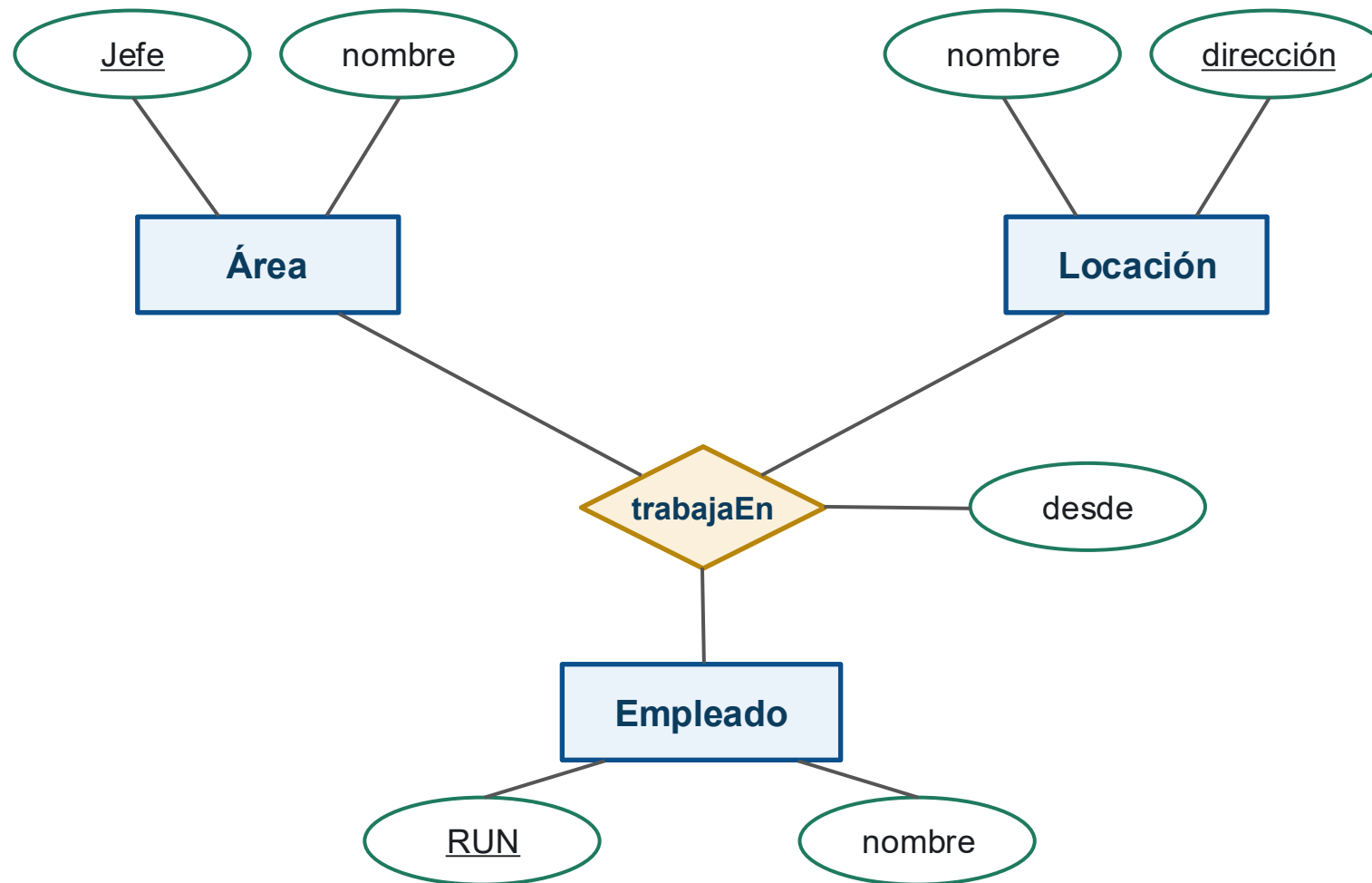
Volvamos al problema de modelar la empresa y sus empleados.

Al diagrama se le quiere añadir una relación que indique si un área tiene sede en una locación y cuántas oficinas tiene en la sede.



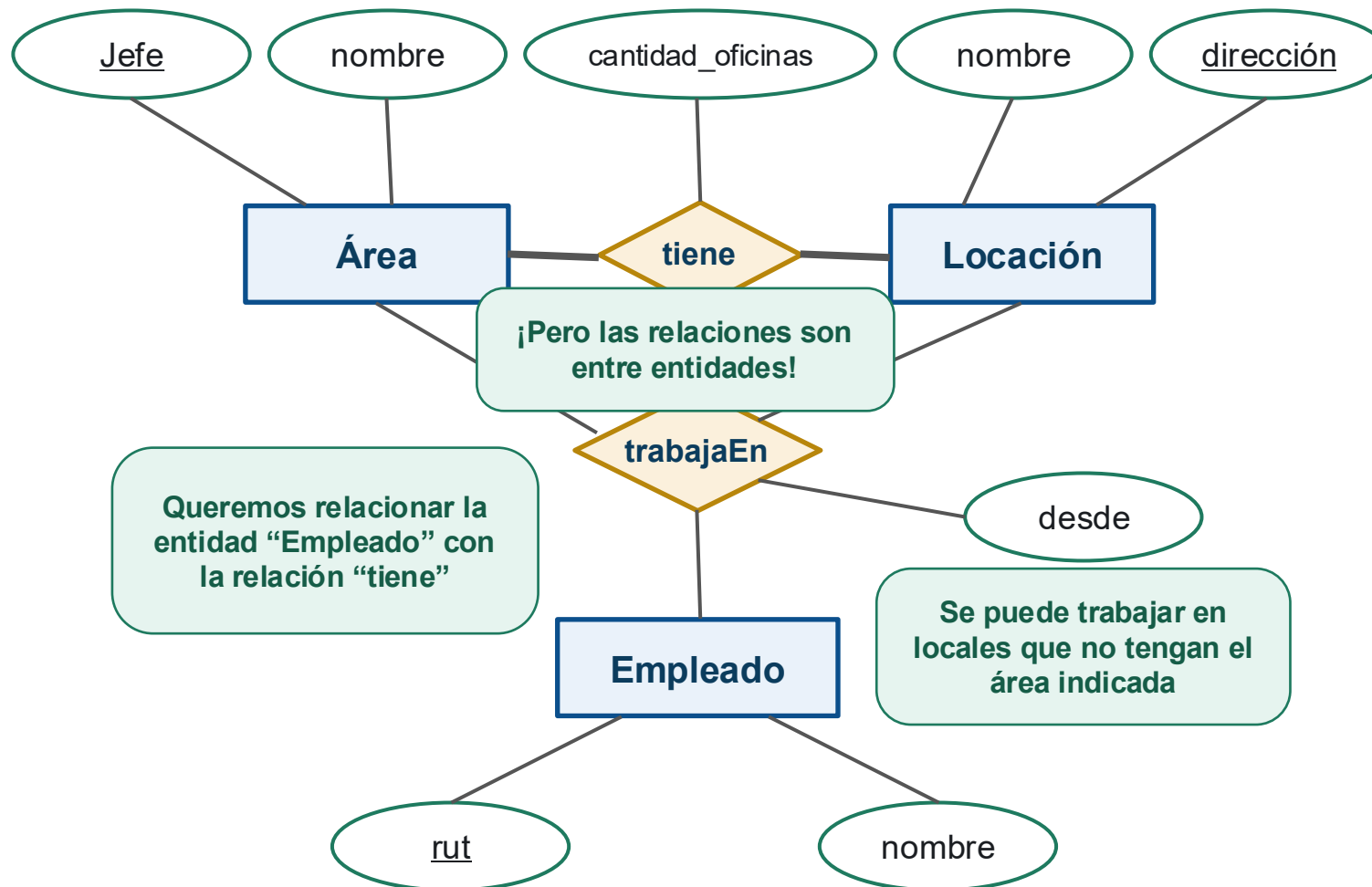
# Agregación

Queremos registrar las locaciones que posee un área y su cantidad de oficinas.



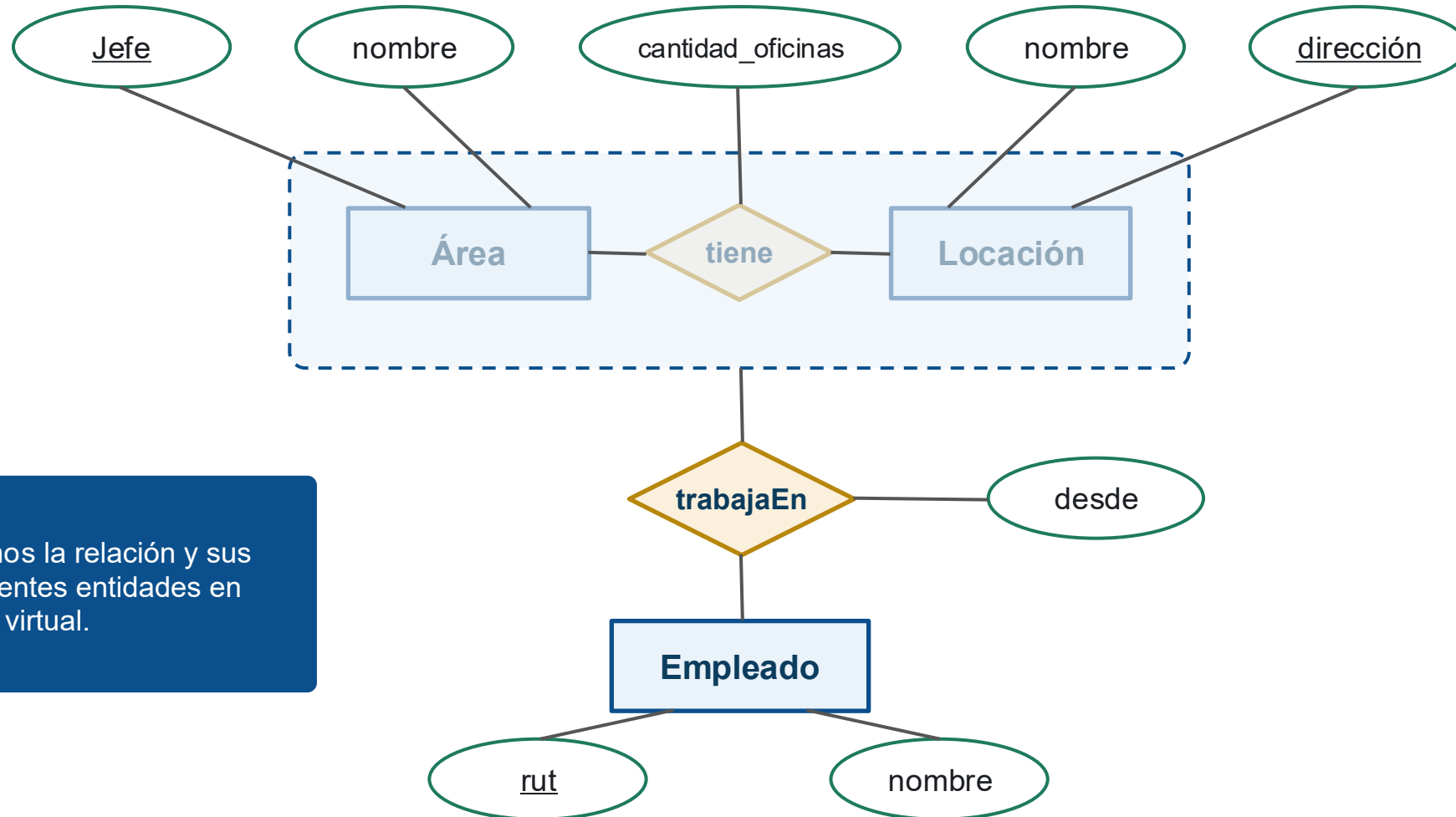
# Agregación

Queremos registrar las locaciones que posee un área y su cantidad de oficinas.



# Agregación

Queremos registrar las locaciones que posee un área y su cantidad de oficinas.

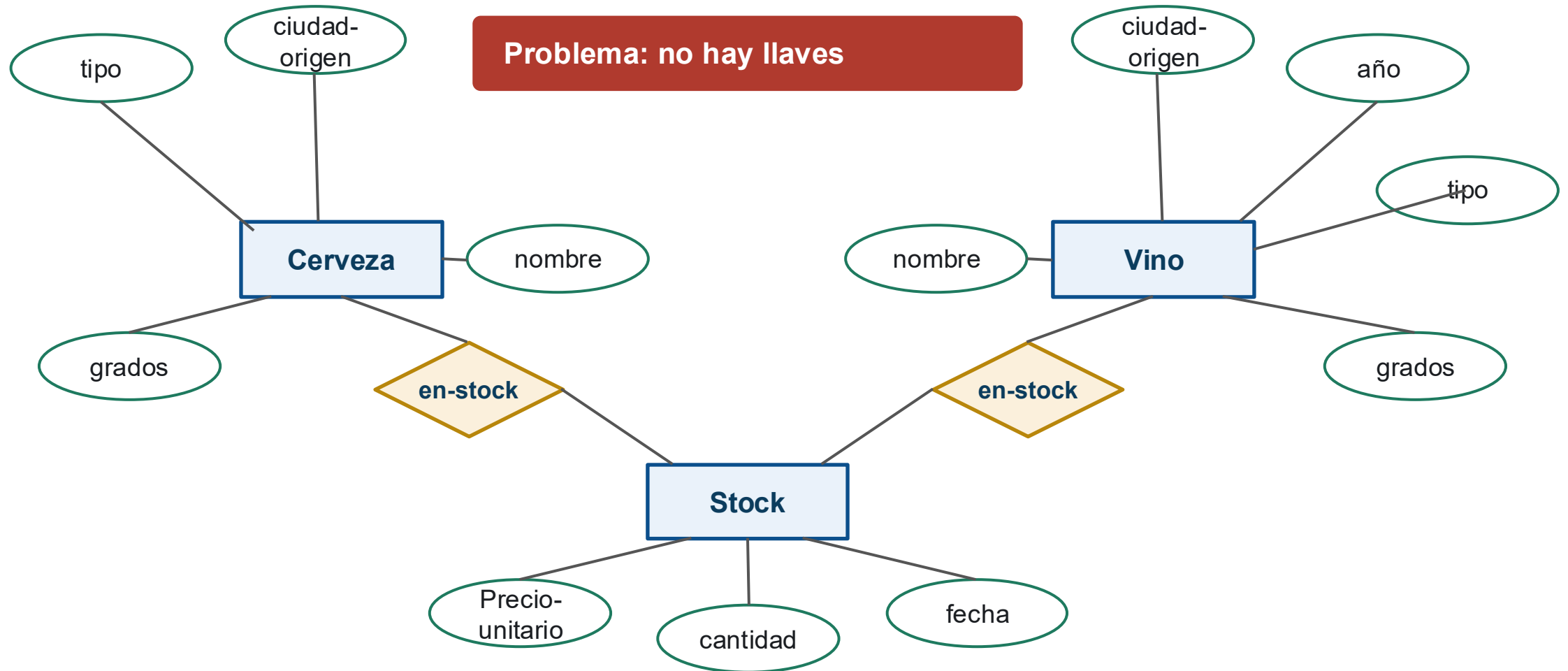


Encapsulamos la relación y sus correspondientes entidades en una entidad virtual.

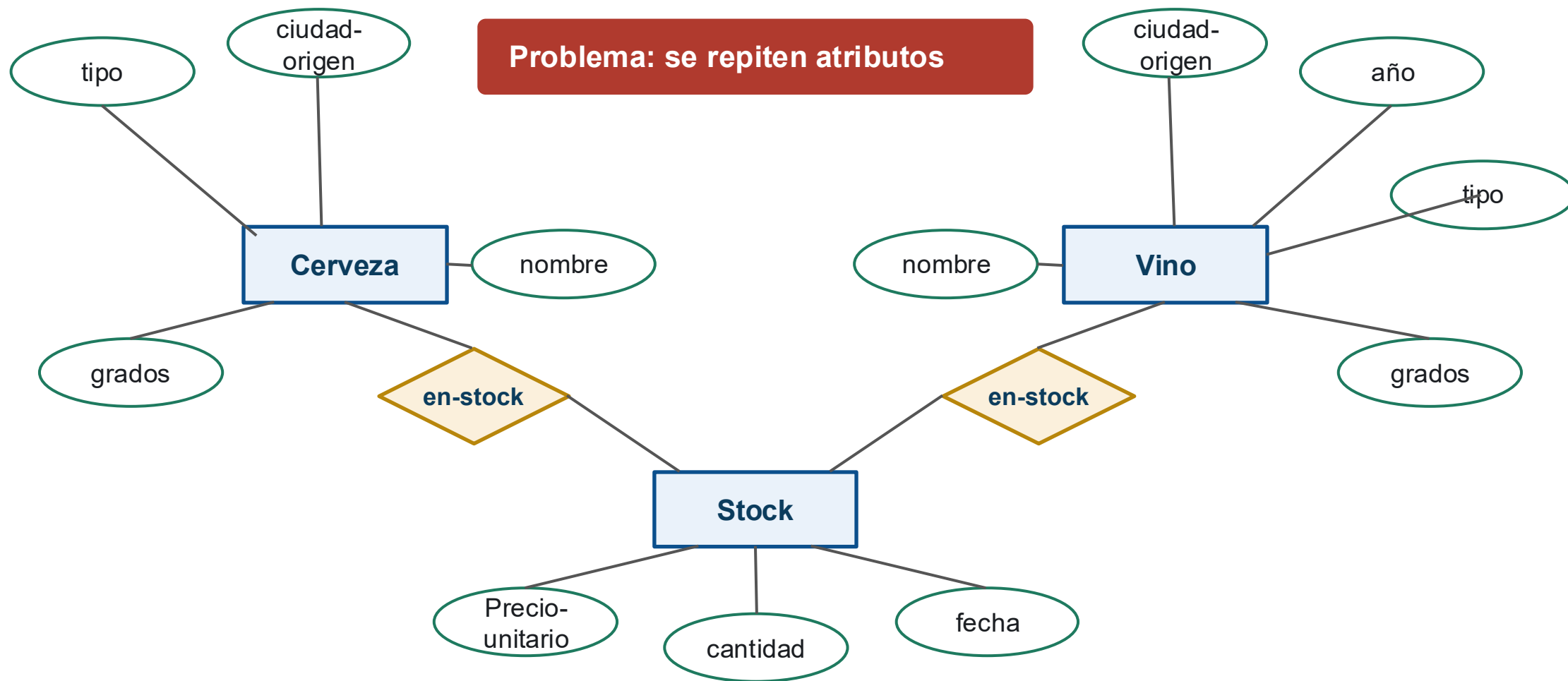
# Ejemplo: Modelando vinos y cervezas

**Problema:** Vendemos vinos y cervezas. Cada vino tiene nombre, año, tipo, grados y ciudad-origen. Cada cerveza tiene nombre, ciudad-origen, tipo, grados. Vinos y cervezas tienen un precio unitario y una cantidad "en stock" cada día.

# Modelando vinos y cervezas

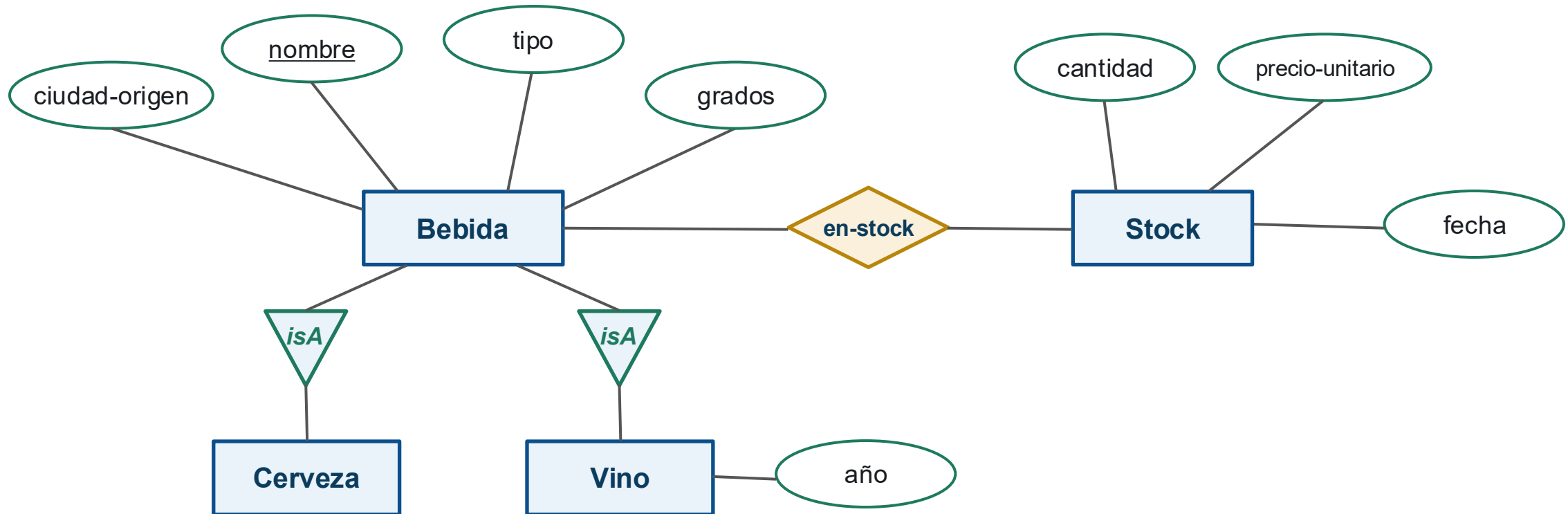


# Modelando vinos y cervezas



# Modelando vinos y cervezas

## Usando jerarquía de clases

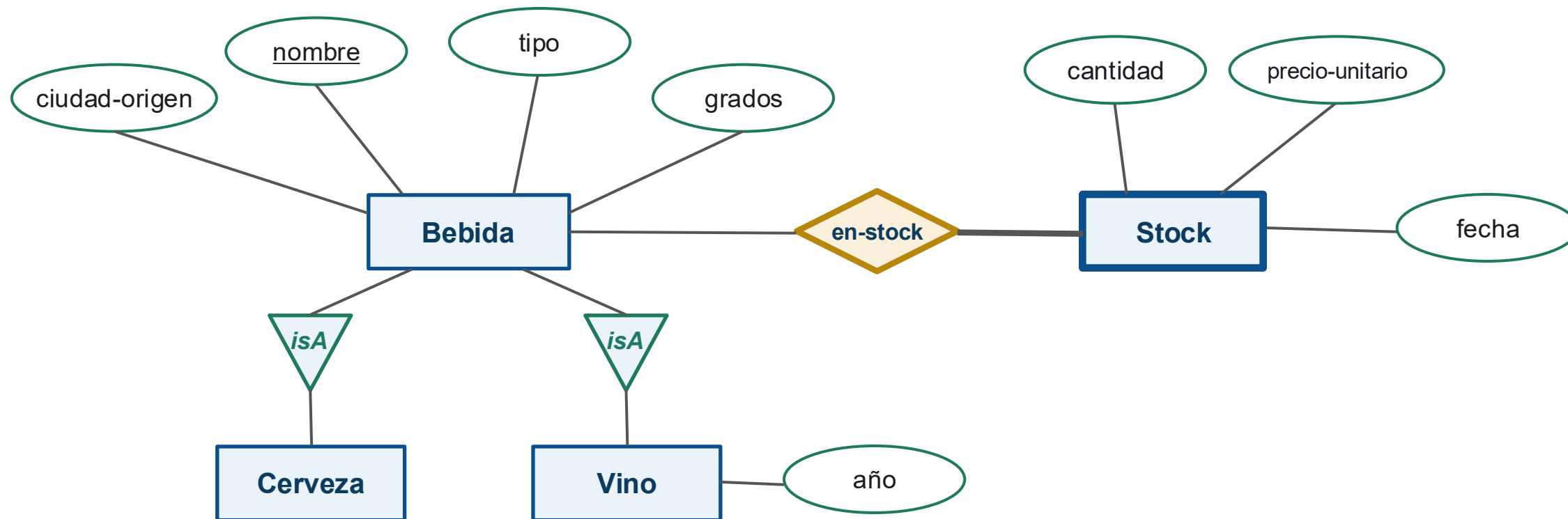


Problema: llave de stock



# Modelando vinos y cervezas

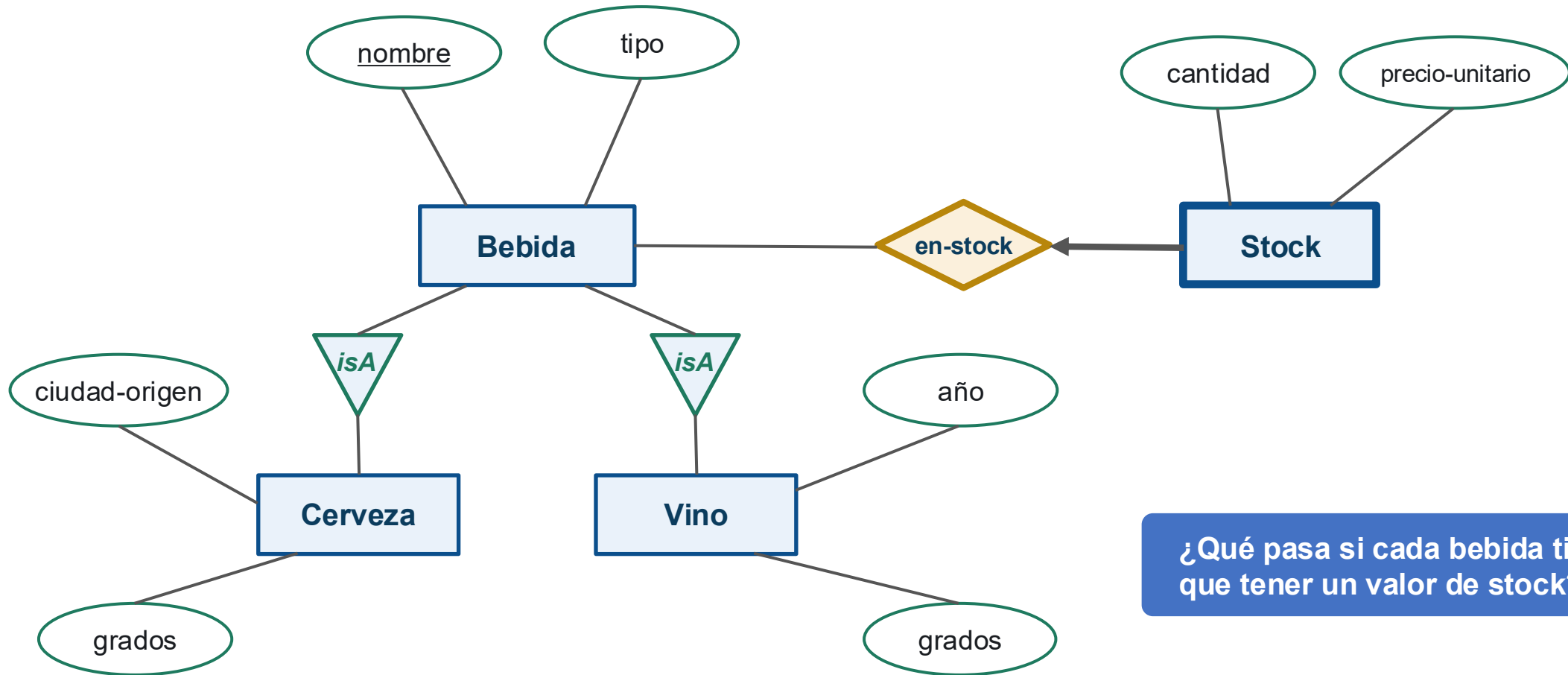
## Usando jerarquía de clases



Problema: multiplicidades

# Modelando vinos y cervezas

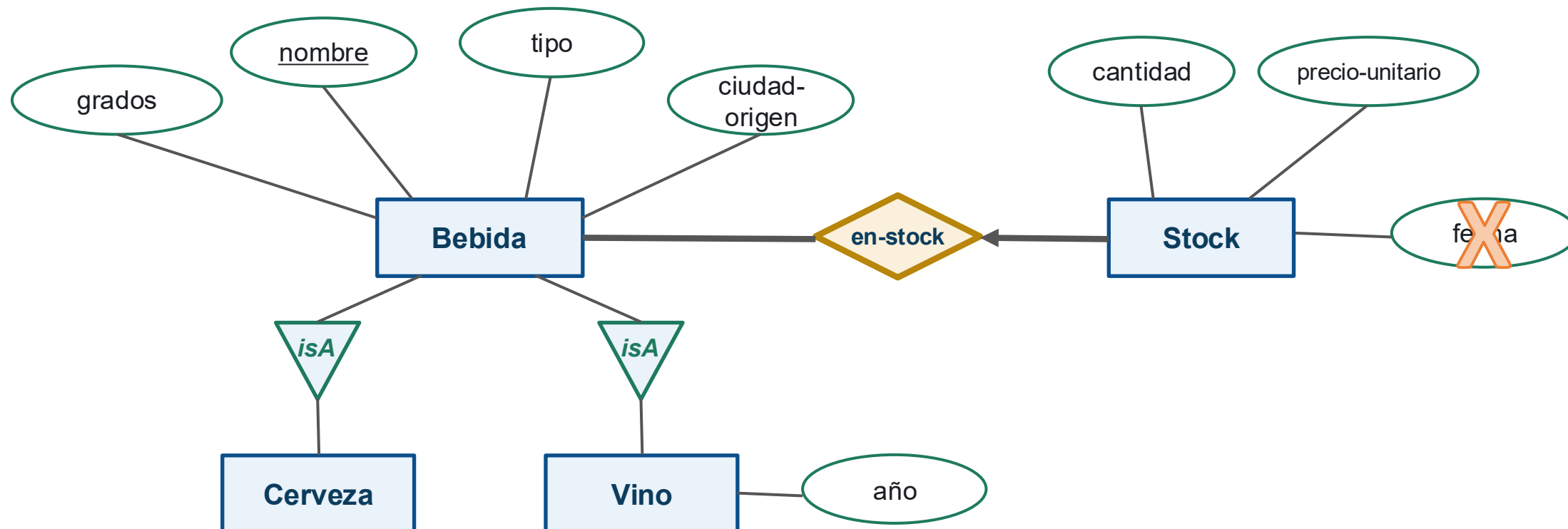
## Con multiplicidades



# Modelando vinos y cervezas

## Con multiplicidades

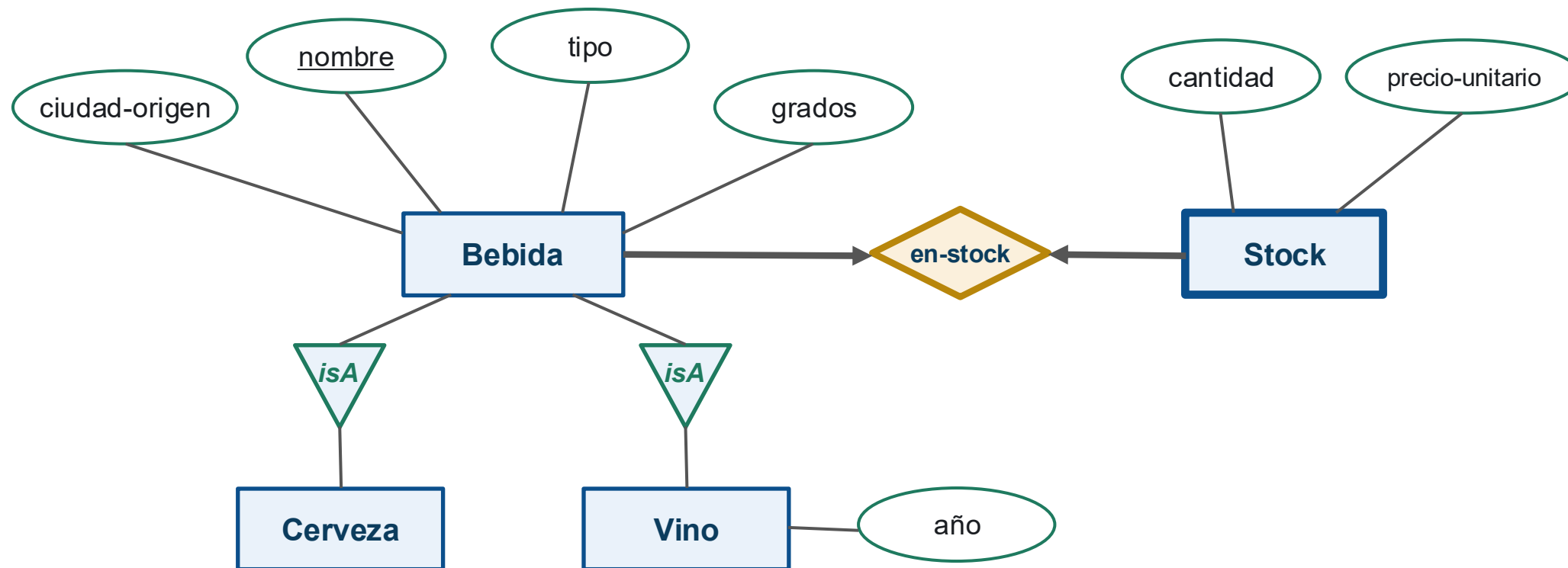
¿Qué pasa si solo se guarda el stock actual?



¿Qué pasa si se guarda el stock actual?

# Modelando vinos y cervezas

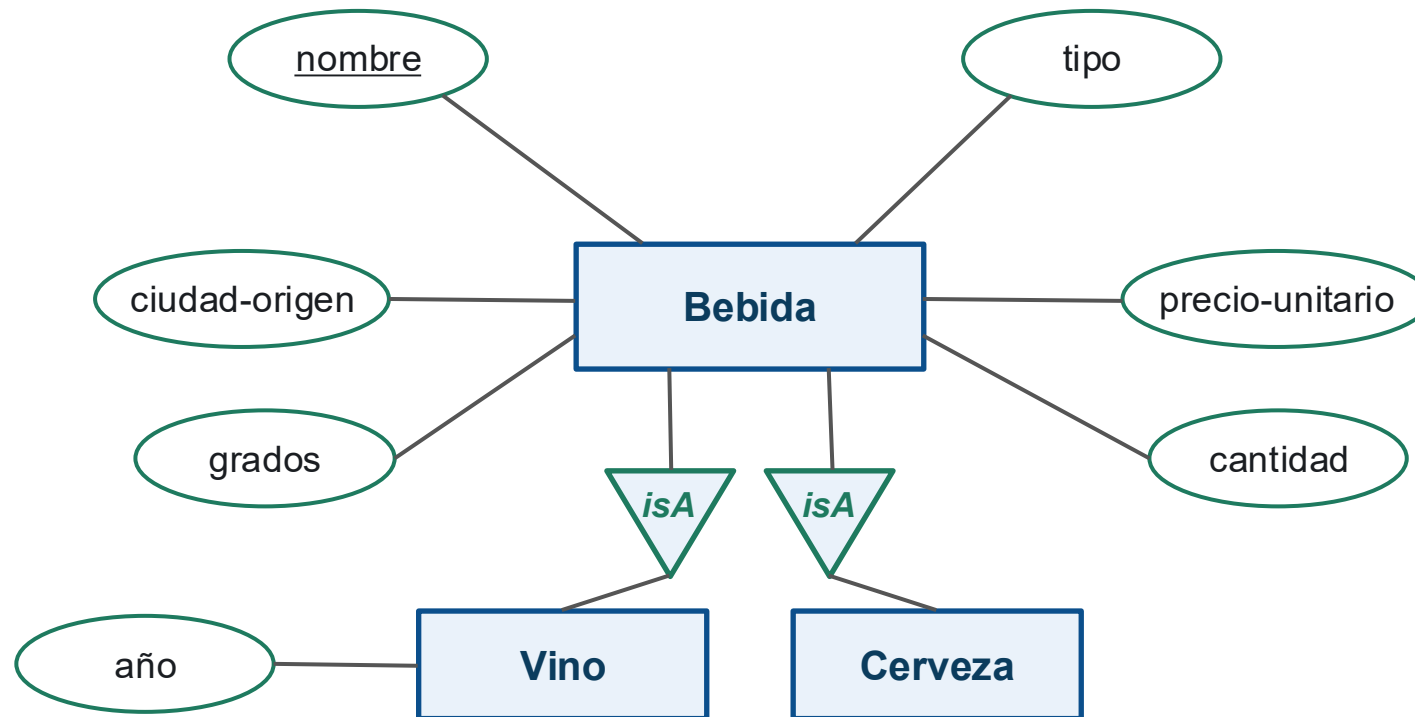
Con solo stock actual



Hay un problema: ¡no hay llave parcial!

# Modelando vinos y cervezas

Con solo stock actual



# Referencia

Capítulo 2, libro guía del curso.